

# RTL Design Sherpa

# Reed-Solomon Codec Hardware Architecture Specification 0.1 (draft)

**October 20, 2026**

# Table of Contents

1 Reed Solomon Has Index....12

2 Document Information....13

2.1 Reed-Solomon Codec Hardware Architecture Specification....13

2.2 Revision History....14

2.3 Document Purpose....15

2.4 Intended Audience....16

2.5 Related Documents....17

3 Purpose and Scope....18

3.1 Purpose....18

3.2 Scope....19

3.3 Where this document stands....20

4 Document Conventions....21

4.1 Terminology....21

4.2 Naming....22

4.3 Notation....23

4.4 Decision markers....24

5 Definitions and Acronyms....25

6 Use Cases....27

6.1 Memory controller ECC....28

6.2 Serial link forward error correction....29

6.3 Storage erasure coding....30

6.4 Standalone codec on a fabric port....31

6.5 Memory-to-memory job engine....32
7 Key Features....33
7.1 Decided (PRD section 3)....33
7.2 Configurable, not yet chosen (chapter 6.2)....35
7.3 Explicitly not features....36
8 System Context....37
8.1 An endpoint codec, not a mid-stream insert....37
8.1.1 Figure 2.1: Where the codec sits....37
8.2 The core inside a consumer....39
8.3 The standalone tops....40
8.4 Dependencies on the rest of the repository....41
9 Block Diagram....42
9.0.1 Figure 3.1: Codec block diagram....42
9.1 The encoder core....43
9.2 The decoder core....44
9.3 The GF layer....45
9.4 The adapters....46
9.5 Hierarchy....47
10 Data Flow....48
10.1 Encoder....48
10.2 Decoder....49
10.2.1 Figure 3.2: Decoder timeline, one symbol per cycle....49
10.3 Block boundaries....51
10.4 Multiple symbols per beat....52

11 Solver Options....53
12 Core Interface....55
12.1 Clock and reset....56
12.2 rs_encoder_core....57
12.3 rs_decoder_core....59
12.4 Timing rules....61
12.5 Erasures (ENABLE_ERASURES, PRD D5 TBD)....62
13 AXI-Stream Adapter....63
13.1 Parameters....64
13.2 Behaviour....65
13.3 Erasure sideband (decoder intake, ERASURE_SUPPORT = 1)....66
14 AXI4 Job Adapter....67
14.1 Job interface....68
14.2 AXI4 master ports....69
14.3 Parameters....70
14.4 Behaviour....71
15 Register Block....72
16 Clock and Reset....74
16.1 Single clock....74
16.2 Reset....75
16.3 Clock gating....76
16.4 Frequency....77
17 Throughput....78
17.1 Sustained rate....79

17.2 What scales with S.....81
18 Latency.....82
18.1 Encoder.....83
18.2 Decoder.....84
18.3 Buffer sizing.....85
19 Resources.....86
19.1 GF primitives per core, reference profile.....87
19.2 Storage.....88
19.3 Order of magnitude.....89
19.4 Scaling.....90
20 System Requirements.....91
20.1 The consumer provides.....92
20.2 The consumer gets.....93
20.3 Environment.....94
21 Parameter Configuration.....95
21.1 Code.....96
21.2 Interface.....98
21.3 Open decisions in one place.....99
22 Candidate Profiles.....100
23 Verification Strategy.....102
23.1 Golden model.....102
23.2 Test matrix.....103
23.3 Test structure.....105
23.4 Formal.....106

23.5 Coverage....107
24 Synthesis and Implementation....108
24.1 Characterisation plan....108
24.2 Implementation notes....109
25 Fields and Polynomials....110
25.1 Why error correction needs a finite field....110
25.2 What a field is....111
25.3 GF(2): the two-element field....112
25.4 Polynomials over GF(2) and irreducibility....113
25.5 Building GF(2^m)....114
25.6 The GF(2^4) log/antilog table....115
25.7 Worked multiplies in GF(2^4)....116
25.8 A worked reduction....117
25.9 The symbol view....118
25.10 Hardware tie: a register is a field element....119
26 From Field to Code....120
26.1 Codewords as polynomials over GF(2^m)....120
26.2 The generator polynomial....121
26.3 Why RS is maximum-distance-separable....122
26.4 Shortened codes....123
26.5 Worked construction of RS(15,11)....124
26.6 Systematic encoding....125
26.7 Worked systematic encode....126
27 Syndromes, the Key Equation, and Correction....129

27.1 The received polynomial....129
27.2 Syndromes....130
27.3 The key equation....131
27.4 Chien search....132
27.5 The Forney formula....133
27.6 Erasure math....134
27.7 Worked decode of RS(15,11)....135
27.7.1 Computing the syndromes....135
27.7.2 Peterson solve for t = 2....136
27.7.3 Chien search in the example....136
27.7.4 Error evaluator and Forney....137
27.7.5 Verification....138
27.8 The decoder’s final verdict....139
27.9 From math to hardware....140
28 The Parameters n, k, t, m....141
28.1 m: the symbol width....142
28.2 n: the block length....143
28.3 k: the user data length....144
28.4 t: the error-correction capability....145
28.5 b: the first-root offset....146
28.6 S: symbols per beat....147
28.7 Parameter summary....148
28.8 Built profiles....149
28.9 Exact relations....151

28.10 Choosing the parameters: a worked example....152
29 Every Stage: Math to Pseudocode....153
29.1 Encoder (gf_lfsr_encoder, wrapped by rs_encoder_core)....154
29.2 Syndrome unit (syndrome_unit, gf_syndrome_cell)....155
29.3 Key-equation solver riBM (key_equation_solver_ribm, ribm_pe)....156
29.4 Key-equation solver Euclid (key_equation_solver_euclid)....158
29.5 Chien search (chien_search)....160
29.6 Forney evaluator (forney_evaluator)....161
29.7 Erasure path (rs_erasure_unit)....162
29.8 Correction and verdict (rs_decoder_core)....164
29.9 Trace: RS(15,11) revisited....166

# List of Figures

Figure 2.1: Where the codec sits....36
Figure 3.1: Codec block diagram....40
Figure 3.2: Decoder timeline, one symbol per cycle....47

# List of Tables

Table 1: Table 0.1: Related documents....17
Table 2: Table 1.1: Definitions and acronyms....25
Table 3: Table 2.1: Use cases and the profile and adapters each implies....32
Table 4: Table 2.2: Repository dependencies....39
Table 5: Table 3.1: The two key-equation solvers....51
Table 6: Table 4.1: Clock and reset....54
Table 7: Table 4.2: Encoder core ports....55
Table 8: Table 4.3: Decoder core ports....57
Table 9: Table 4.4: AXI-Stream mapping....61
Table 10: Table 4.5: AXI-Stream adapter parameters....62
Table 11: Table 4.6: Job fields....66
Table 12: Table 4.7: AXI4 job adapter parameters....68
Table 13: Table 4.8: Register block outline....70
Table 14: Table 5.1: Sustained throughput....77
Table 15: Table 5.1a: Measured cycles per block....77
Table 16: Table 5.2: Scaling with symbols per beat....79
Table 17: Table 5.3: Encoder latency....81
Table 18: Table 5.4: Decoder latency....82
Table 19: Table 5.5: GF primitive counts, RS(255,239), t = 8, m = 8, S = 1....85
Table 20: Table 5.6: Storage....86
Table 21: Table 6.1: Code parameters....94
Table 22: Table 6.2: Interface parameters....96
Table 23: Table 6.3: Open PRD decisions and their defaults....97
Table 24: Table 6.4: Candidate profiles....98
Table 25: Table 6.5: Verification matrix....101
Table 26: Table 6.6: Characterisation steps....106
Table 27: Table 7.1: Operations in GF(2)....110
Table 28: Table 7.2: Powers of alpha in GF(2^4) with p(x) = x^4 + x + 1....113
Table 29: Table 7.3: Sample multiplications in GF(2^4)....114
Table 30: Table 7.4: Field operations and their hardware forms....117
Table 31: Table 7.5: Generator polynomial g(x) for RS(15,11)....122
Table 32: Table 7.6: Data symbols for the RS(15,11) worked example....124
Table 33: Table 7.7: LFSR remainder evolution during encoding....124
Table 34: Table 7.8: Final RS(15,11) codeword....125
Table 35: Table 7.9: Received word with two injected errors....133
Table 36: Table 7.10: Syndromes of the received word....134

Table 37: Table 7.11: Error locator polynomial coefficients.....134
Table 38: Table 7.12: Chien search over the 15 positions.....135
Table 39: Table 7.13: Forney error magnitudes.....136
Table 40: Table 7.14: Post-correction syndromes.....136
Table 41: Table 7.15: Mathematical operations mapped to decoder blocks.....138
Table 42: Table 7.16: Reed-Solomon parameters and their RTL names.....146
Table 43: Table 7.17: Two shipped Reed-Solomon profiles.....147
Table 44: Table 7.18: Parameter equations.....149
Table 45: Table 7.19: From mathematics to hardware blocks.....151

# 1 Reed Solomon Has Index

**Generated:** 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 2 Document Information

## 2.1 Reed-Solomon Codec Hardware Architecture Specification

| Property | Value |
|---|---|
| Document Title | Reed-Solomon Codec Hardware Architecture Specification |
| Version | 0.3 (draft) |
| Date | September 29, 2026 |
| Status | Draft – decisions D2, D3, D5 and D10 of the PRD are open; this document carries them as parameters |
| Classification | Open Source - MIT License |

## 2.2 Revision History

| Version | Date | Author | Description |
|---|---|---|---|
| 0.1 | 2026-09-29 | RTL Design Sherpa | First draft, written before any RTL exists. Captures what the PRD has decided (valid/ready core with optional AXIS and AXI4 adapters, symbol width as a parameter with the bus a multiple of it, riBM solver with a Euclidean alternative, scrambler behind an enable, endpoint placement, BCH out of scope) and carries the undecided items – correctable-symbol count and profile, shortening, erasures, first consumer – as parameters with candidate values. Numbers are analytic, from the FUB catalog; none are measured. |
| 0.2 | 2026-10-20 | RTL Design Sherpa | Block diagrams redrawn to the repo element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key under each figure. No technical content changed. |
| 0.3 | 2026-10-20 | RTL Design Sherpa | Section (h2) headings now open a new page, the pipeline default. No technical content changed. |

## 2.3 Document Purpose

This Hardware Architecture Specification (HAS) is the high-level view of the Reed-Solomon codec – what it is, where it sits, what it presents at its boundaries, what it costs and how it is configured. It is the document an integrator reads before instantiating the core in a memory controller, a compute engine or a link, and the document a verification engineer reads to know what must be proven.

It covers:

- the encoder and decoder cores and their valid/ready interfaces
- the optional AXI-Stream and AXI4 adapters for standalone use
- the configuration parameters and the candidate code profiles
- throughput, latency and resource estimates and how they scale
- integration requirements, verification strategy and synthesis plan

Micro-architecture – the GF arithmetic, the riBM and Euclidean arrays, the Chien and Forney datapaths, FSMs and signal-level timing – lives in the companion Micro-Architecture Specification (MAS) at `../reed_solomon_mas/`, written against the landed RTL. `docs/rs_fub_catalog.md` remains the bottom-up build plan the MAS checks the tree against.

## 2.4 Intended Audience

- System architects deciding where error correction sits in a datapath
- Integrators instantiating the core or one of its adapters
- Verification engineers planning the golden-model comparison
- Anyone judging whether an RS code, and which one, fits a channel

## 2.5 Related Documents

*Table 0.1: Related documents*

| Document | Location | Content |
|---|---|---|
| Product Requirements | `../../PRD.md` | the decision table (D1-D12) and the candidate profiles |
| FUB catalog | `../rs_fub_catalog.md` | every block bottom-up with what it instantiates |
| Architecture sketch | `../rs_architecture_sketch.md` | block diagram and reuse map |
| References | `../../References/README.md` | the standards and papers, with source and licence |
| Handbook | `vault/handbook/design/` | valid/ready contract, reset and clocking, filelists, generated-RTL discipline |

# 3 Purpose and Scope

## 3.1 Purpose

The Reed-Solomon codec adds symbol-level forward error correction to a data path. An encoder appends 2t parity symbols to every block of k data symbols; a decoder, given the n = k + 2t received symbols, corrects up to t symbol errors – or e errors and f erasures with 2e + f <= 2t – and reports what it did. A symbol is m bits, so a single corrupted symbol may carry m corrupted bits: RS is the code for channels whose errors come in bursts and for memories whose failures take out a whole device or a whole nibble at once.

The component exists so that a Sherpa consumer – a memory controller, a compute engine, a serial link – can drop a proven codec into its own datapath instead of building one. That is why the deliverable is a CORE with plain valid/ready streaming ports at both ends (PRD D9): the consumer keeps its own interfaces and the codec is a block inside them.

## 3.2 Scope

In scope:

- `rs_encoder_core` and `rs_decoder_core`: systematic RS(n, k) over GF(2^m), parameterised in m, t and shortening
- the GF(2^m) primitive layer they are built from, written so the planned BCH component can reuse it
- optional boundary adapters for standalone use: AXI-Stream at either end, or an AXI4 read or write engine with a job interface at either end, chosen independently per end
- an optional scrambler / pseudo-randomizer stage behind a parameter
- the configuration registers, status and counters of the standalone tops
- verification against a software golden model, and FPGA characterisation

Out of scope (PRD section 6 and D7):

- BCH: its own component, `ecc-ip/bch/`
- soft-decision, list and folded RS decoding
- concatenation with convolutional or LDPC codes, and interleaving beyond a symbol-address permutation at the boundary
- any mid-stream insertion of the codec into a fabric (chapter 2.3 says why)

## 3.3 Where this document stands

No RTL exists at revision 0.1. Everything here is derived from the PRD's decisions, the FUB catalog's block hierarchy and the standards in `References/`. Where a number appears it is analytic – a cell count from the algorithm, a latency from the pipeline – and is labelled so. Measured throughput, timing and resources arrive with the first synthesis run (chapter 6.5) and will replace the estimates in chapter 5.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 4 Document Conventions

## 4.1 Terminology

- **Symbol**: an m-bit field element. `SYMBOL_WIDTH` = m. All arithmetic is in GF(2^m).
- **Block**: n symbols on the coded side, k on the data side. The core delimits blocks with a `last` flag on the final symbol of each.
- **Beat**: one transfer on a valid/ready port; carries `SYMBOLS_PER_BEAT = DATA_WIDTH / SYMBOL_WIDTH` symbols.
- **Profile**: one fully-resolved parameter set – m, t, n, k, first root, primitive polynomial, scrambler – typically named after the standard it matches (chapter 6.3).
- **Core**: `rs_encoder_core` or `rs_decoder_core`, valid/ready at both ends.
- **Adapter**: the optional AXIS or AXI4 boundary around a core.

## 4.2 Naming

Module and signal names follow the repository conventions (`vault/handbook/design/`): `aclk` / `aresetn`, active-low asynchronous reset through the `ALWAYS_FF_RST` / `RST_ASSERTED` macros, `r_` registers and `w_` wires internally, valid/ready per the house contract. Parameters are upper case. Generated files carry a “Generated by” banner and live in `rtl/generated/`.

## 4.3 Notation

- n, k, t, m as above; 2t = n - k parity symbols.
- alpha: the primitive element of GF(2^m) defined by the primitive polynomial p(x); b: the first root of the generator polynomial g(x) = (x + alpha^b)(x + alpha^(b+1)) … (x + alpha^(b+2t-1)).
- Lambda(x): error-locator polynomial; Omega(x): error-evaluator polynomial; S_i: syndromes.
- Counts quoted “for the reference profile” mean RS(255,239), t = 8, m = 8, one symbol per beat, unless a table says otherwise.

## 4.4 Decision markers

Where the PRD has decided something, this document states it as fact and cites the decision (for example “PRD D11”). Where the PRD has not, the text says **TBD** and names the decision; chapter 6.2 collects every TBD in one table.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 5 Definitions and Acronyms

*Table 1.1: Definitions and acronyms*

| Term | Meaning |
|---|---|
| RS(n, k) | Reed-Solomon code with n coded symbols per block, k of them data |
| GF(2^m) | the Galois field with 2^m elements; symbols and all codec arithmetic live in it |
| t | the number of symbol errors the code corrects per block; 2t = n - k |
| erasure | a symbol whose position is known to be unreliable (flagged at the input); costs one parity symbol to fix instead of two |
| shortened code | RS(n', k') with n' < 2^m - 1, derived from the full-length code by treating the missing leading symbols as zero (DVB's RS(204,188) from RS(255,239)) |
| systematic | the k data symbols pass through unchanged and the 2t parity symbols follow them |
| syndrome | S_i = R(alpha^(b+i)), the received polynomial evaluated at a generator root; all zero if and only if the block is a codeword |
| key equation | the relation the solver inverts to find Lambda(x) and Omega(x) from the syndromes |
| riBM | reformulated inversionless Berlekamp-Massey, the default key-equation solver (Sarwate and Shanbhag 2001) |
| Euclid / ME | the modified Euclidean algorithm in its inversionless systolic form (Shao et |

| Term | Meaning |
|---|---|
| | al. 1985), the alternative solver |
| Chien search | evaluating Lambda at every position to find the roots, hence the error locations |
| Forney | computing each error's magnitude from Omega and Lambda' at the located root |
| dual basis | the alternative field representation CCSDS uses for its symbols; a fixed linear map at the boundary |
| scrambler / PRBS | the binary LFSR sequence a standard XORs onto the data for energy dispersal or sync; not part of the RS code |
| AXIS | AXI4-Stream |
| PRD | Product Requirements Document (`../../PRD.md`) |
| MAS | Micro-Architecture Specification (companion, `../reed_solomon_mas/`) — per-block signal-level detail of the landed RTL |
| FUB | functional unit block |

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

# 6 Use Cases

The codec is an endpoint block (chapter 2.3). Its use cases are the endpoints a Sherpa design has or may grow, each matched to one of the PRD's candidate profiles. None is yet a committed consumer – that is PRD decision D10 – so this chapter is the menu the first consumer chooses from.

## 6.1 Memory controller ECC

A memory controller encodes on its write path and decodes on its read path. Both cores sit inside the controller's own datapath behind its scheduler, which already delimits bursts and so provides the block boundary. Symbols of 8 bits map a byte-wide device failure to one correctable symbol; t is chosen from the device's failure model. This is the shape the pumice / scoria / andesite family would take if it adopted RS over its present SECDED, and it is the use case that argues most strongly for the bare valid/ready core.

## 6.2 Serial link forward error correction

A transmitter encodes each frame and a receiver decodes it; the scrambler sits outside the encoder on the line side. Two profiles come straight from standards in `References/`: DVB's RS(204,188) over GF(2^8) and the CCSDS RS(255,223) telemetry code with its dual-basis symbol representation. The 802.3 RS-FEC codes over GF(2^10) are the high-rate end of this use case and drive the multi-symbol-per-beat option.

## 6.3 Storage erasure coding

A controller writes parity across devices and reconstructs a lost device on read. This is the erasure-only use of RS (Plank 1997): known-bad positions, no Chien search, any n <= 2^m - 1. It exercises the erasure input (PRD D5) and the AXI4 job adapter (memory-to-memory), and it is the one use case that may want an encoder without a full decoder.

## 6.4 Standalone codec on a fabric port

The core wrapped in AXI-Stream at both ends, inserted where a stream already has block boundaries (TLAST) and room for the parity – a framer's output, a DMA's stream port. This is the AXIS/AXIS adapter pairing and the simplest way to bring the block up in a system.

## 6.5 Memory-to-memory job engine

The core wrapped in an AXI4 read engine and an AXI4 write engine, driven by jobs from its register block or a descriptor stream: encode this buffer into that one, or decode. A DMA with a transform. This is the AXI4/AXI4 pairing and the natural test vehicle for the erasure use case.

*Table 2.1: Use cases and the profile and adapters each implies*

| Use case | Profile | Adapter pairing | Erasures | Notes |
|---|---|---|---|---|
| Memory controller ECC | m = 8, t per device model | none (bare cores) | optional | the shape the core was designed for |
| Serial link FEC | DVB RS(204,188) or CCSDS RS(255,223) | none, or AXIS/AXIS | no | scrambler on the line side |
| High-rate link FEC | 802.3 RS(544,514), m = 10 | none | no | needs SYMBOLS_PER_BEAT > 1 |
| Storage erasure coding | m = 8 or 16, erasure-only | AXI4/AXI4 | yes, primary | Plank's construction |
| Standalone stream codec | any | AXIS/AXIS | optional | bring-up vehicle |
| Memory-to-memory job | any | AXI4/AXI4 | optional | test vehicle for erasures |

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

# 7 Key Features

## 7.1 Decided (PRD section 3)

- **Valid/ready core at both ends** (D9). `rs_encoder_core` and `rs_decoder_core` present the house streaming contract: valid, ready, data, a `last` flag on the final symbol of a block; the decoder adds erasure flags in and a per-block status out. A consumer instantiates the core and nothing else.
- **Optional adapters, selectable per end** (D9). `INTAKE_IF` and `OUTLET_IF` each `NONE` (default), `AXIS` or `AXI4`, independently, so AXIS-in/AXI4-out and the other pairings are one core with different adapters generated. Only the adapters selected exist in the netlist.
- **Symbol width is a code property** (D1). `SYMBOL_WIDTH` = m is its own parameter (default 8). `DATA_WIDTH` must be an integer multiple of it, checked at elaboration; `SYMBOLS_PER_BEAT = DATA_WIDTH / SYMBOL_WIDTH` is the throughput factor (D6). A bus that does not divide fails to elaborate rather than straddling symbols across beats.
- **Two key-equation solvers** (D11). `KES_ALGO = "RIBM"` (default): the reformulated inversionless Berlekamp-Massey array, 3t + 1 cells, shortest critical path. `KES_ALGO = "EUCLID"`: the modified Euclidean array, 2t cells, the textbook algorithm in systolic form. Same ports, same result up to a common scale; one generate in the decoder core selects them.
- **Scrambler behind a parameter** (D12). `ENABLE_SCRAMBLER` generates the standard's PRBS stage with its polynomial, seed and placement as parameters; the OFF build carries no LFSR logic and is a tested configuration.
- **Generated field constants** (R3). The primitive polynomial, generator coefficients and log/antilog tables are emitted by a Python generator into `rtl/generated/` from the profile parameters; no field constant is a literal in the datapath.
- **Every block reports** (R2). The decoder never silently passes a failed block: per-block status carries ok / corrected count / uncorrectable, and running counters feed the register block.
- **Golden-model verification** (R4). Every profile is checked against `reedsolo` or `galois` on random data, error patterns up to t and beyond t (to prove the uncorrectable flag), and erasures where enabled.

## 7.2 Configurable, not yet chosen (chapter 6.2)

- t, the correctable-symbol count, and with it n and k (D2)
- shortening (D3) – full length or a standard's shortened block
- erasure decoding present or absent (D5)
- encoder-only builds for transmit-only or RAID-write consumers (D4)
- per-profile generator conventions: first root b, dual basis (D8)
- the first consumer, which fixes all of the above (D10)

## 7.3 Explicitly not features

- BCH (its own component), soft-decision or list decoding, interleaving beyond a boundary permutation, concatenation with other codes.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

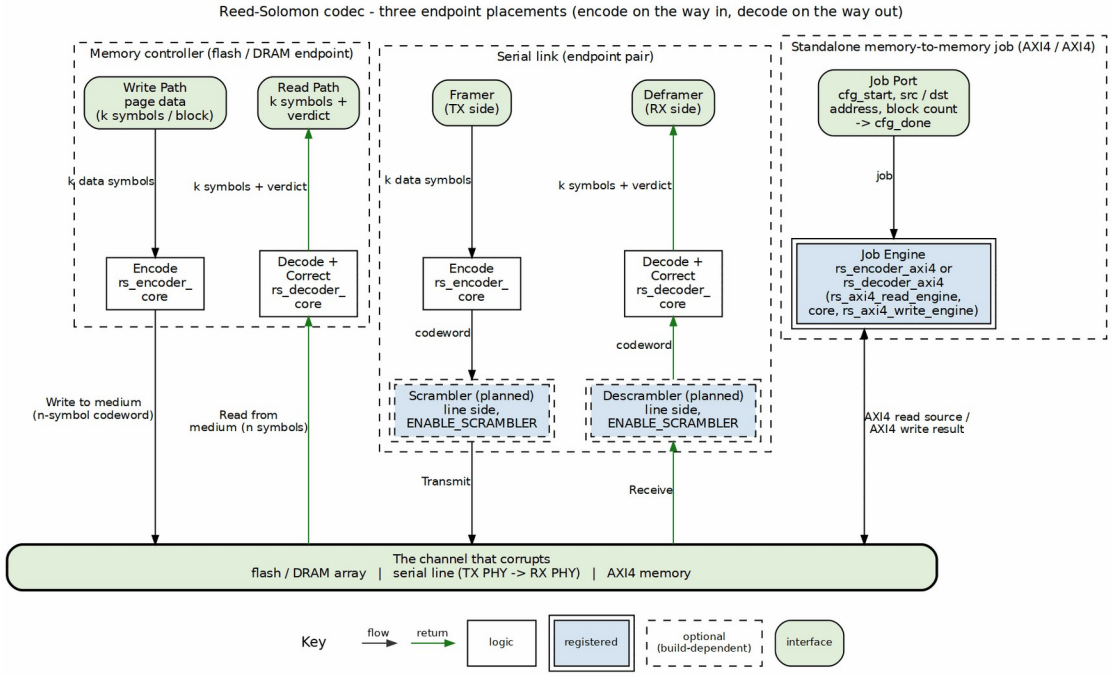
# 8 System Context

## 8.1 An endpoint codec, not a mid-stream insert

RS protects a block, not a wire. The encoder must see k symbols before it can emit the first parity symbol, and the decoder must see all n before it can correct any of them, because the syndromes are a function of the whole block. The code therefore lives where a block boundary already exists and where the extra 2t symbols have somewhere to go. Every standard in `References/` places it at an endpoint: the CCSDS transmitter and ground receiver, the DVB modulator and set-top demodulator, the 802.3 PCS at each end of a link, the CD read channel, the storage controller at write and at read.

Dropping the codec into the middle of a fabric fails on three counts: the parity changes the byte count, so the downstream must run faster or the upstream must leave gaps; block alignment must survive every hop, which is why every RS standard sits behind a sync marker, a fixed frame or a page boundary; and the correction belongs immediately after the channel that corrupts, not several repackings later.

### 8.1.1 Figure 2.1: Where the codec sits




*Where the codec sits*

Three endpoint shapes: a memory controller with an encoder on its write path and a decoder on its read path; a serial link pair with the scrambler on the line side (`ENABLE_SCRAMBLER`, PRD D12, is planned and not in the RTL yet, so the figure draws it dashed); and the standalone memory-to-memory job engine built from the AXI4 adapters.

## 8.2 The core inside a consumer

With valid/ready at both ends the core is a block in the consumer's own datapath. The consumer supplies block boundaries (its scheduler, framer or job control already knows where they are), absorbs the 2t/k rate change, and keeps its own external interfaces. Nothing in the consumer's protocol handling changes; what changes is that k symbols go in and n come out (or n in, k out with a status).

## 8.3 The standalone tops

When there is no consumer datapath to sit inside – a codec on a fabric port, a memory-to-memory transform – the standalone tops wrap the core in adapters (chapters 4.2 and 4.3) and a register block (chapter 4.4), and the existing AXI-Stream and AXI4 monitors observe the boundaries onto the monitor bus.

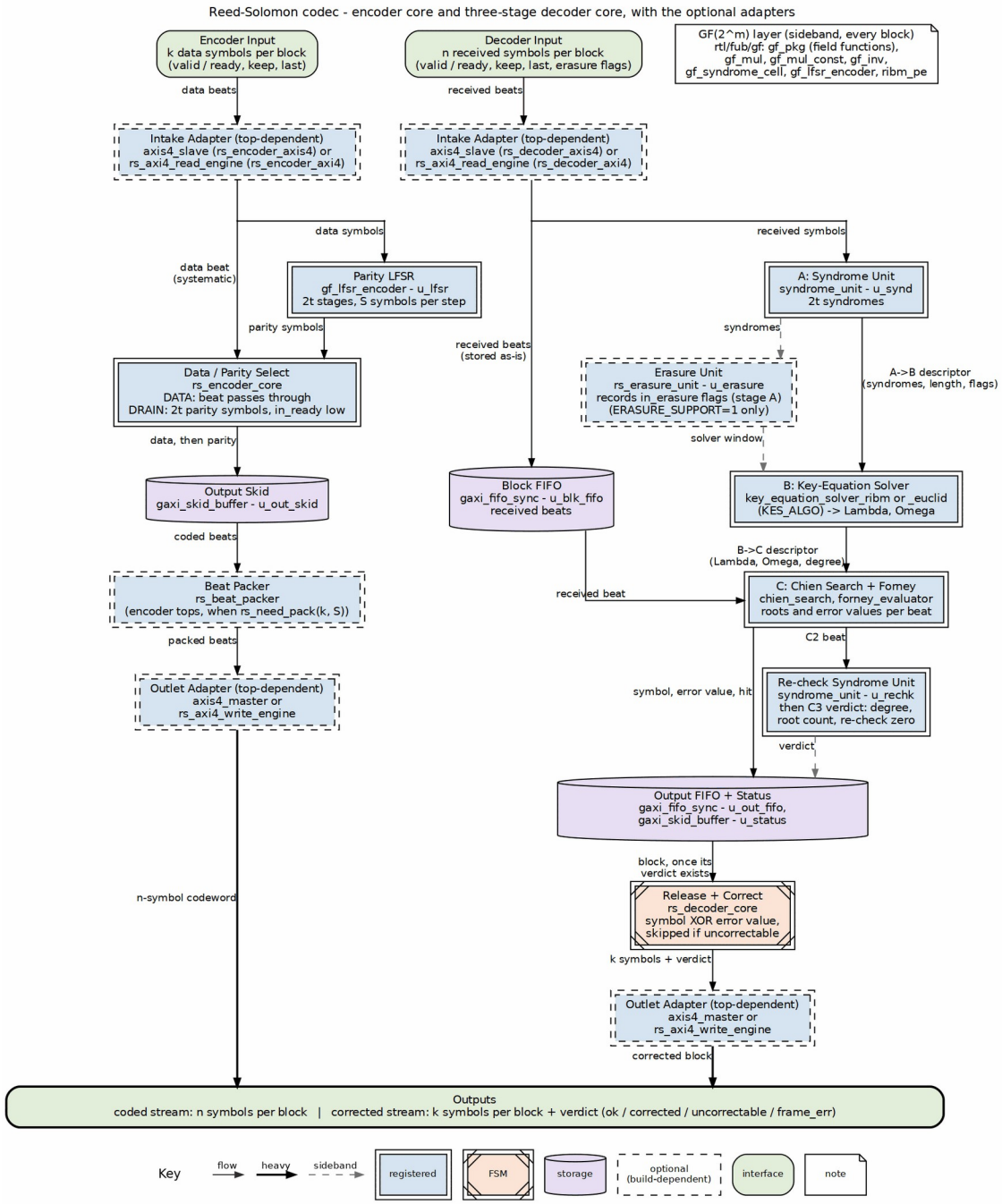
## 8.4 Dependencies on the rest of the repository

*Table 2.2: Repository dependencies*

| Dependency | Used for |
|---|---|
| `rtl/amba/gaxi`: `gaxi_skid_buffer`, `gaxi_fifo_sync` | stage decoupling; the decoder's block buffer |
| `rtl/common`: `counter_bin`, `shifter_beat_pack`, `shifter_lfsr_galois` | positions and iterations; symbol packing; the scrambler |
| `rtl/amba/axis4`: `axis4_slave`, `axis4_master` (+ `_monlite`) | AXIS adapters |
| `rtl/amba/axi4`: `axi4_master_rd`, `axi4_master_wr` | AXI4 adapters' timing wrappers |
| `rtl/amba/ocelus`: `axis_monitor_lite` | boundary observation onto monbus |
| `bin/peakrdl_generate.py` + `utility-ip/converters` | APB-to-cpuif path | the register block and its APB attachment |
| `dma-ip/stream/rtl/fub`: `axi_read_engine`, `axi_write_engine` | the model for the AXI4 job engines |
| `reedsolo`, `galois` (PyPI, MIT) | DV golden model |

# 9 Block Diagram

### 9.0.1 Figure 3.1: Codec block diagram




*Codec block diagram*

The two cores, the shared GF layer, and the optional adapters that wrap a core for standalone use.

## 9.1 The encoder core

Three blocks in a straight line, all inside `rs_encoder_core`. The parity generator is `gf_lfsr_encoder` (`u_lfsr`): a 2t-stage shift register over GF(2^m) with the generator polynomial's coefficients as constant multipliers on its taps. Each accepted beat steps it by the beat's symbol count (the S single steps are unrolled), so a whole beat enters in one cycle. The data / parity select passes each data beat straight through, and once `in_last` arrives it drains the 2t parity symbols S at a time with `in_ready` held low, raising `last` on the final one; the drain leaves the LFSR all-zero, so there is no clear. A `gaxi_skid_buffer` on the output decouples the consumer. There is no separate unpack or pack stage inside the core; the encoder tops add `rs_beat_packer` when k is not a multiple of S, so the codeword's only partial beat is its last.

## 9.2 The decoder core

`rs_decoder_core` is three stages joined by descriptor skids, so one block is received while the previous one is solved and the one before that is corrected. Stage A writes every received beat into the block FIFO (`gaxi_fifo_sync`) and steps `syndrome_unit`, whose 2t cells each accumulate one syndrome by Horner's rule; on the last symbol a descriptor (length, frame error, all-zero flag, syndromes) goes to stage B. Stage B runs the key-equation solver – the riBM array by default, the modified Euclidean array when `KES_ALGO` selects it – to produce the error-locator and error-evaluator polynomials; a clean or mis-framed block bypasses it. With `ERASURE_SUPPORT=1`, `rs_erasure_unit` records the erasure flags in stage A and transforms and combines the polynomials in stage B. Stage C loads `chien_search` and `forney_evaluator`, then walks the block one beat per cycle against the block FIFO; the walked beat is registered, fed to a second syndrome unit (the re-check) and pushed into the output FIFO as {received symbol, error value, hit}. One cycle after the last beat the verdict (degree, root count, zero derivative, re-check) goes to the status skid. A block leaves only when both its data and its verdict are present, and the error values are applied on the way out only when the block is not uncorrectable, so an uncorrectable block leaves exactly as it arrived.

## 9.3 The GF layer

`gf_pkg` holds the field arithmetic as constant functions of (m, primitive polynomial): alpha powers, the multiply function and the elaboration checks. Each module builds its own tables from it; there is no generator. The primitives built on it are `gf_mul_const` (a constant multiply, an XOR network), `gf_mul` (full multiply), `gf_inv` (inverse), plus the composite cells `gf_syndrome_cell`, `gf_lfsr_encoder` and `ribm_pe`. The block counts are in chapter 5.3.

## 9.4 The adapters

The adapters come from the top that wraps the core. `rs_encoder_axis4` and `rs_decoder_axis4` put the house `axis4_slave` and `axis4_master` wrappers around the core, with TLAST as the block boundary and the decoder's erasure flags on the intake. `rs_encoder_axi4` and `rs_decoder_axi4` are memory-to-memory job engines: `rs_axi4_read_engine` feeds the core and `rs_axi4_write_engine` drains it, sequenced by a job port (`cfg_start`, source and destination address, block count, `cfg_done`). Chapter 4 specifies each.

## 9.5 Hierarchy

The complete bottom-up hierarchy – every block, what it does and every component it instantiates with counts – is `docs/rs_fub_catalog.md`. This chapter names the blocks; the catalog is the authority on their contents.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 10 Data Flow

## 10.1 Encoder

One beat in, one beat out, with the output stream 2t/k longer than the input.

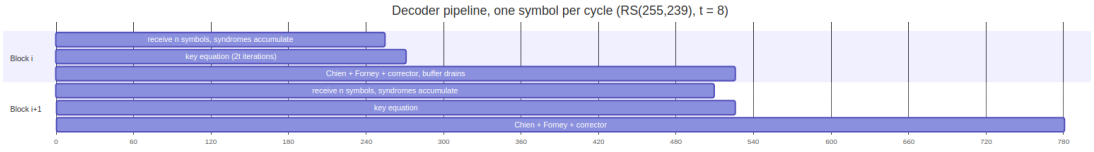
1. A beat of `SYMBOLS_PER_BEAT` data symbols arrives with `in_valid`; the core accepts it when the encoder can take it (`in_ready`).
2. Each symbol enters the LFSR (updating the parity registers) and is passed straight to the output mux – systematic encoding means the data are not delayed by the code, only by the pipeline.
3. On the k-th symbol (`in_last`), the mux switches to the parity registers and emits the 2t parity symbols over the following cycles, then raises `out_last`; the LFSR clears for the next block.
4. While parity drains, `in_ready` is low: the encoder cannot start a new block until the previous block's parity has left. A consumer that wants full-rate back-to-back blocks provides an input FIFO of at least 2t symbols upstream, or accepts the 2t-cycle gap.

## 10.2 Decoder

One beat in, one beat out, with a block of latency in between.

1. Received symbols are written to the block buffer and fed to the syndrome cells. Erasure flags, if enabled, are recorded alongside each symbol.
2. On `in_last` the syndromes are complete. If all are zero the block is clean: the solver is skipped and the buffer drains through the corrector unchanged, status = ok.
3. Otherwise the key-equation solver runs 2t iterations. If the resulting locator degree exceeds t, or is zero, the block is already known to be uncorrectable.
4. Chien search steps through positions 0 .. n-1, one per cycle, while the buffer reads out the same positions. At each root, Forney supplies the error value. Every corrected symbol, data and parity alike, is fed to a second syndrome unit; the received data symbols and their corrections go to the output FIFO as separate fields.
5. On the last position the verdict is final: uncorrectable if the root count differs from the degree, if a derivative was zero at a root, or if the re-computed syndromes are not all zero. That last check is what catches the blocks with more than t errors whose wrong locator still has the right number of roots; the degree and root checks alone miscorrect some of them.
6. The block is released only once its verdict exists. Corrections are applied on the way out, and not at all when the block is uncorrectable, so an uncorrectable block leaves exactly as it arrived. The status ports hold the verdict for every beat of the block and `out_last` marks its k-th.
7. The next block's symbols arrive during steps 3-6 and the one after that is being solved while this one is walked; the buffers are sized for that overlap, so the decoder sustains one block per n cycles.

### 10.2.1 Figure 3.2: Decoder timeline, one symbol per cycle




*Decoder timeline*

For the reference profile the first data beat of a block emerges about 2n/S + 2t cycles after its first beat arrived (the block is received, solved, and walked to its verdict before release), and the block then drains in k/S cycles while the next block is walked, so sustained throughput is one block every n/S cycles – measured at 252 to 257 cycles per RS(255,239) block at S = 1, 64.5 at S = 4 and 33 at S = 8 (n/S = 32).

## 10.3 Block boundaries

The core does not count symbols to find a block: `in_last` is the boundary, and the core checks it. A block shorter or longer than the configured n (or k on the encoder side) is reported as a framing error in status and the core resynchronises on the next `in_last`. A shortened code (`N_SYMBOLS` < 2^m - 1) is handled by the core treating the missing leading symbols as zero in the syndromes and by offsetting the Chien positions; the consumer sees only the shortened block.

## 10.4 Multiple symbols per beat

With `SYMBOLS_PER_BEAT = S > 1` every stage works a beat per cycle: the encoder LFSR advances by the beat's symbol count (S single steps unrolled, the state after the count-th taken), the syndrome cells take S Horner steps, and the Chien and Forney cells evaluate S positions from one set of registers, each lane a constant multiply away from the beat's first position. The solver works on the 2t syndromes and is unaffected. Blocks are still delimited by `last`; `keep` is low-aligned and only a block's last beat may be partial.

Where the partial beats fall differs between the cores. The encoder passes data beats through as received and appends the parity in ceil(2t/S) beats, so its output may carry a partial beat at the end of the data and another at the end of the parity; a consumer that needs contiguous packing puts a beat packer at the outlet, which is the adapters' job (chapter 4.2, 4.3). The decoder drops the parity lanes, so its output is k symbols in ceil(k/S) beats with only the final beat partial.

# 11 Solver Options

The key-equation solver is the one block with two implementations, selected by KES_ALGO (PRD D11). They have identical ports and produce the same error-locator and error-evaluator polynomials up to a common nonzero scale factor – which is harmless, because Chien search finds the same roots and Forney takes the ratio of the two polynomials.

*Table 3.1: The two key-equation solvers*

| | **riBM ("RIBM", default)** | **modified Euclidean ("EUCLID")** |
|---|---|---|
| Algorithm | reformulated inversionless Berlekamp-Massey, Sarwate and Shanbhag 2001 | Sugiyama's Euclidean algorithm with cross-multiplication in place of division (Shao et al. 1985), every polynomial kept top-aligned so no shifter is needed |
| Array | 3t + 1 processing elements, 2 registers each | four register arrays: R and Q of 2t + 1 symbols, the shifted locator pair of 2t + 3 |
| GF multipliers | 2 per element: 6t + 2 (50 at t = 8) | 2 per coefficient of each pair: 8t + 8 (72 at t = 8) |
| Iterations | exactly 2t | data-dependent, t + 1 .. 2t (2t + 1 at t <= 2 when a leading syndrome is zero), plus one cycle for the finishing check |
| Evaluator produced | the high half of S(x)Lambda(x); Forney uses X^(1 - b - 2t) | the textbook S(x)Lambda(x) mod x^2t; Forney uses X^(1 - b). The Forney block's OMEGA_HIGH_HALF constant follows KES_ALGO; nothing else in the decoder changes |

| | riBM (`"RIBM"`, default) | modified Euclidean (`"EUCLID"`) |
|---|---|---|
| Critical path per iteration | one multiply and one XOR, no feedback across the array | cross-multiply feeding a degree comparison that drives the swap: longer |
| Control | discrepancy select and gamma update | two nominal degree counters, the normalise / cross / swap rule |
| Readability | dense; the reformulation is not obvious from the textbook | the textbook algorithm, recognisable step by step |
| Fallback | – | the degree-computationless variant (Baek and Sunwoo 2006) removes the counters at the cost of a wider element |

riBM is the default because its per-iteration critical path is shorter and does not cross the array, which is what sets the decoder's clock at high t. Euclid is kept for readability and as the cross-check: both are bit-exact against `dv/tbclasses/rs_model.py`, which runs both solvers on every block of its validation and requires identical decoder output (900 of 900 blocks on six profiles), and the decoder core test runs the same profiles with either solver selected.

Nothing else in the decoder knows which solver produced Lambda. Syndromes, Chien, Forney, the block buffer and the corrector are the same modules in both builds.

# 12 Core Interface

Both cores present the house valid/ready streaming contract (`vault/handbook/design/valid-ready-contracts.md`) at each end: `valid` and `data` are held stable until `ready`, `ready` may depend on `valid`, `valid` must not depend on `ready`. Widths are in bits; `S` is `SYMBOLS_PER_BEAT` and `m` is `SYMBOL_WIDTH`.

## 12.1 Clock and reset

*Table 4.1: Clock and reset*

| Port | Dir | Width | Description |
|---|---|---|---|
| `aclk` | in | 1 | one clock for the whole core |
| `aresetn` | in | 1 | active-low asynchronous reset, per the repository convention |

## 12.2 rs_encoder_core

*Table 4.2: Encoder core ports*

| Port | Dir | Width | Description |
|---|---|---|---|
| in_valid | in | 1 | a beat of data symbols is offered |
| in_ready | out | 1 | the core takes it this cycle |
| in_data | in | S x m | S data symbols, symbol 0 in the low bits |
| in_keep | in | S | which symbols of the beat are present: low-aligned, all ones except possibly the final beat of a block |
| in_last | in | 1 | this beat carries the k-th data symbol of the block |
| out_valid | out | 1 | a beat of coded symbols is offered |
| out_ready | in | 1 | the consumer takes it |
| out_data | out | S x m | S coded symbols: the data beats as received, then the 2t parity symbols in ceil(2t/S) beats |
| out_keep | out | S | present-symbol mask of the beat: partial at the end of the data and at |

| Port | Dir | Width | Description |
|---|---|---|---|
| | | | the end of the parity when S divides neither k nor 2t |
| `out_last` | out | 1 | this beat carries the n-th coded symbol of the block |
| `frame_err` | out | 1 | pulse: a block ended with other than k data symbols; it was still encoded as given |

## 12.3 rs_decoder_core

*Table 4.3: Decoder core ports*

| Port | Dir | Width | Description |
|---|---|---|---|
| in_valid / in_ready | in / out | 1 | as the encoder |
| in_data | in | S x m | S received symbols |
| in_keep | in | S | present-symbol mask |
| in_last | in | 1 | this beat carries the n-th received symbol |
| in_erase | in | S | (only with ENABLE_ERASURES) symbol is a known erasure |
| out_valid / out_ready | out / in | 1 | as the encoder |
| out_data | out | S x m | S corrected data symbols; parity lanes are dropped, so the block is k symbols in ceil(k/S) beats |
| out_keep | out | S | present-symbol mask, low-aligned, partial only on the block's final beat |
| out_last | out | 1 | this beat carries the k-th data symbol of the block |
| out_status_ok | out | 1 | held for every |

| Port | Dir | Width | Description |
|---|---|---|---|
| | | | beat of the block: it had no errors |
| out_status_corrected | out | log2(t)+1 | held for every beat of the block: symbols corrected (0 .. t) |
| out_status_uncorrectable | out | 1 | held for every beat of the block: correction failed; the data are exactly as received |
| out_status_frame_err | out | 1 | held for every beat of the block: block length was not n; the block passes through uncorrected, its first length - 2t symbols as data |

## 12.4 Timing rules

- `in_last` is the block boundary. The core does not infer boundaries by counting; it checks the count against n (or k) and reports a mismatch.
- The encoder holds `in_ready` low while it drains the parity of the previous block, ceil(2t/S) cycles (chapter 3.2); a consumer wanting no gap buffers that many beats upstream.
- `keep` masks are low-aligned. A partial beat anywhere but a block's last is a framing error on the decoder and a count mismatch on the encoder.
- The decoder accepts the next block while it is still correcting the previous one, up to the depth of its block buffer; `in_ready` falls only when the buffer is full, which under sustained back-pressure on `out_ready` it will be.
- Status outputs describe the block being emitted and are valid on every beat of it, first to last; a consumer may sample them with `out_last` or with the first beat. A block is not released until its verdict is final, so an uncorrectable verdict is never preceded by altered symbols.
- Reset clears block state and the syndrome and parity registers; the block buffer is not reset (its contents are indexed by pointers that are).

## 12.5 Erasures (ENABLE_ERASURES, PRD D5 TBD)

When enabled, `in_erase` marks symbols the consumer already knows are bad (a failed device, a dropped packet). Each erasure costs one parity symbol instead of two: the decoder corrects e errors and f erasures with 2e + f <= 2t. When disabled the port does not exist and the erasure-locator logic is not generated.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 13 AXI-Stream Adapter

Generated at an end when its *_IF parameter is "AXIS". It is the house axis4_slave (intake) or axis4_master (outlet) timing wrapper – the same modules the bridge uses at its AXI-Stream ports – with the core's stream mapped onto the AXI-Stream signals as follows.

*Table 4.4: AXI-Stream mapping*

| AXIS signal | Core signal | Notes |
|---|---|---|
| TVALID / TREADY | valid / ready | one-to-one |
| TDATA[DATA_WIDTH-1:0] | data | S symbols per beat |
| TKEEP[S*m/8-1:0] | keep | byte keep; symbols are whole bytes when m = 8, else TKEEP is per symbol byte-group and TSTRB is not used |
| TLAST | last | the block boundary |
| TUSER (intake, decoder) | in_erasure[S-1:0] | erasure flags, one per symbol lane, only when ERASURE_SUPPORT = 1; otherwise the consumer's TUSER passes through the intake unused |
| TUSER (outlet, decoder) | status | {frame_err, uncorrectable, corrected[..], ok}, valid with TLAST |
| TID, TDEST | – | passed through unchanged from intake to outlet when both ends are AXIS; tied off otherwise |

## 13.1 Parameters

*Table 4.5: AXI-Stream adapter parameters*

| Parameter | Default | Meaning |
|---|---|---|
| `AXIS_SKID_DEPTH` | 2 | skid depth in the wrapper (2..8) |
| `AXIS_ID_WIDTH`, `AXIS_DEST_WIDTH` | 0 | pass-through sideband widths |
| `AXIS_MONITOR` | 0 | 1 selects the `_monlite` wrapper variant, which attaches `axis_monitor_lite` to the port and emits onto the monitor bus |

## 13.2 Behaviour

The adapter adds no protocol semantics: TLAST is the block boundary the core requires, and the rate change is visible as more beats out than in on the encoder (or fewer on the decoder). A stream that arrives without TLAST at block boundaries cannot be decoded and is reported as a framing error on every block.

## 13.3 Erasure sideband (decoder intake, ERASURE_SUPPORT = 1)

The decoder wrapper takes a per-beat `in_erasure[S-1:0]` port, one flag per symbol lane, valid with the beat. The flags are spliced ABOVE the consumer's TUSER through the intake skid – the skid is built with `AXIS_USER_WIDTH = AXIS_USER_WIDTH + S` (a zero consumer width still costs one guard bit) and the core-facing side slices the top S bits back off – so the flags stay beat-aligned with their data under any backpressure, which a sideband registered outside the skid cannot guarantee. With `ERASURE_SUPPORT = 0` the port does not exist and the intake is the pre-erasure wrapper, bit for bit.

One integration rule this forces: the flags ride TUSER through the skid, so any configuration that zeroes `AXIS_ID_WIDTH` or `AXIS_DEST_WIDTH` must run the fixed `axis4_slave` (per-field tuser unpack; amba BUG-038, the 8-branch concatenation shifted tuser right whenever a zero-width guard bit preceded it). The DV grid sweeps `(ID, DEST) in {(4, 2), (0, 0)}` for both wrappers.

# 14 AXI4 Job Adapter

Generated at an end when its `*_IF` parameter is `"AXI4"`. The intake form is a read engine: a job names a source address and a byte count, the engine issues AXI4 read bursts and presents the returned data as the core's input stream, inserting `last` every n (decoder) or k (encoder) symbols. The outlet form is a write engine: it drains the core's output stream into AXI4 write bursts at the job's destination address, one block at a time, and reports each block's status in its completion. A job controller sequences the two.

## 14.1 Job interface

Jobs arrive from the register block (chapter 4.4) – source, destination, count, kick – or, with `JOB_IF = "DESC"`, from a small AXI-Stream descriptor port carrying the same fields for chained jobs.

*Table 4.6: Job fields*

| Field | Width | Meaning |
|---|---|---|
| `src_addr` | `AXI_ADDR_WIDTH` | read engine start address (intake = AXI4) |
| `dst_addr` | `AXI_ADDR_WIDTH` | write engine start address (outlet = AXI4) |
| `byte_count` | 32 | bytes to read; must be a whole number of blocks |
| `kick` / `done` | 1 / 1 | start; completion pulse with the job's status summary |
| `cfg_erasure` | `N_SYMBOLS` | decoder only, `ERASURE_SUPPORT = 1`: job-level erasure bitmap, sampled at `cfg_start` and sliced per beat by the same beat index that builds `keep` – the shape an MC known-bad column has (a fixed position set, identical in every block of the job) |

The AXI-Stream adapter's erasure sideband is per-beat because a mid-stream producer knows symbol positions only as the beat passes; the job adapter's is per-job because a memory controller learns a bad column once and applies it to everything it fetches. Both land on the core's same `in_erasure` port.

## 14.2 AXI4 master ports

Each engine presents one AXI4 master (read-only or write-only) through the house `axi4_master_rd` / `axi4_master_wr` timing wrapper: `AXI_ID_WIDTH`, `AXI_ADDR_WIDTH`, `AXI_DATA_WIDTH` (= `DATA_WIDTH`), incrementing bursts up to `cfg_xfer_beats`, no narrow transfers. Responses other than OKAY are counted and reported in the job's done status; the data are still passed.

## 14.3 Parameters

*Table 4.7: AXI4 job adapter parameters*

| Parameter | Default | Meaning |
|---|---|---|
| `AXI_ADDR_WIDTH` | 32 | address width |
| `AXI_ID_WIDTH` | 4 | ID width; the engines use one ID |
| `AXI_MAX_BURST_BEATS` | 16 | burst cap (`cfg_xfer_beats` may lower it at run time) |
| `JOB_IF` | `"REGS"` | `"REGS"` or `"DESC"` |
| `AXI_MONITOR` | 0 | 1 attaches the `_monlite` wrapper variant |

## 14.4 Behaviour

The read engine is modelled on STREAM's `axi_read_engine` and the write engine on its `axi_write_engine` – job valid/address/count in, done strobe out, a burst cap – reduced from STREAM's multi-channel, SRAM-coupled form to one job and a FIFO. Whether that is a reduction of STREAM's modules or a fresh instantiation is decided when the adapter is built. The job controller hands the write engine each block's destination before the block's first symbol reaches it, so a decoder job's output blocks land contiguously with the parity removed.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 15 Register Block

The standalone tops carry a PeakRDL-generated register block, `rs_regs`, attached through the converters' APB-to-cpuif path exactly as STREAM's configuration block attaches its own. The bare cores have no registers: their configuration is parameters, and their status is the per-block status port. The map below is the outline; the RDL is the authority once written.

*Table 4.8: Register block outline*

| Register | Access | Contents |
|---|---|---|
| `ID` | RO | component ID, version, `KES_ALGO`, `ENABLE_*` build flags |
| `PROFILE` | RO | m, t, n, k, first root b, primitive polynomial index of the build |
| `CTRL` | RW | enable, soft reset, scrambler enable (if built), erasure enable (if built) |
| `JOB_SRC`, `JOB_DST`, `JOB_LEN` | RW | job fields (AXI4 ends only) |
| `JOB_KICK` | WO | start the job |
| `JOB_STATUS` | RO | busy, done, last job's status summary, AXI response errors |
| `STAT_BLOCKS` | RO | blocks processed |
| `STAT_CORRECTED_BLOCKS` | RO | blocks with at least one correction |
| `STAT_CORRECTED_SYMBOLS` | RO | total symbols corrected |
| `STAT_UNCORRECTABLE` | RO | blocks flagged uncorrectable |
| `STAT_FRAME_ERR` | RO | blocks with a length mismatch |
| `IRQ_EN`, `IRQ_STATUS` | RW / W1C | interrupt on uncorrectable, on job |

| Register | Access | Contents |
|---|---|---|
| | | done, on frame error |

Registers are accessed by name through the generated regmap in DV and host code, never by offset (`vault/handbook/dv/registers-by-name.md`). Counters are 32-bit, saturating, cleared by `CTRL.soft_reset`.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

# 16 Clock and Reset

## 16.1 Single clock

The core and its adapters run on one clock, `aclk`. A consumer whose data path runs on another clock crosses at the consumer's boundary with the repository's CDC blocks (`rtl/cdc`); the core does not contain a crossing. For a standalone top whose AXI4 memory side runs on a different clock than the stream side, the adapter's timing wrapper is the place to insert the `axi4_cdc_{rd,wr}` pair, as the bridge does for its CDC slave ports; that is a build option for the adapter, not a property of the core.

## 16.2 Reset

`aresetn` is active-low and asynchronous, applied through the `ALWAYS_FF_RST` macros with `RST_ASSERTED` so the polarity follows the repository build (`vault/handbook/design/reset-and-clocking.md`). Reset clears control state, the parity and syndrome registers, position and iteration counters and the status. The block buffer's storage is not reset; its pointers are, so stale contents are never read.

## 16.3 Clock gating

None inside the core. The `_cg` variants of the AXIS and AXI4 wrappers may be selected at the adapters for a standalone top that wants boundary clock gating, following the rest of the repository.

## 16.4 Frequency

The decoder's clock is set by the key-equation solver's per-iteration critical path (one GF multiply plus an XOR for riBM), then by the syndrome and Chien cells (one constant multiply plus an XOR). At m = 8 these are a few LUT levels; chapter 5.3 gives the characterisation plan that turns this into a number per part.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

# 17 Throughput

All figures analytic at revision 0.1.

## 17.1 Sustained rate

Both cores consume one beat per cycle when not back-pressured. With `SYMBOLS_PER_BEAT = S`, that is S symbols, S x m bits, per cycle.

*Table 5.1: Sustained throughput*

| Core | Sustained | Gap | Notes |
|---|---|---|---|
| Encoder | ceil(k/S) beats in per ceil(k/S) + ceil(2t/S) cycles | ceil(2t/S) cycles per block while parity drains, unless the consumer buffers that many beats upstream | output rate is n/k times the input rate |
| Decoder | one block per ceil(n/S) + 1 cycles | the verdict stage is one entry deep, so a block's successor cannot enter the correct stage until its verdict is written; otherwise none, the next block arrives during the previous block's Chien walk | output rate is k/n times the input rate |

Measured in simulation, back-to-back timing at both ends:

*Table 5.1a: Measured cycles per block*

| Profile | S | Encoder, cycles per block | Decoder, cycles per block (steady state) |
|---|---|---|---|
| RS(255,239) | 1 | 255 | 255.5 |
| RS(255,239) | 4 | 64 | 67.5 |
| RS(255,239) | 8 | 32 | 36 |
| RS(204,188) | 8 | 26 | 30 |
| RS(21,19) | 4 | 6 | 10.2 |

| Profile | S | Encoder, cycles per block | Decoder, cycles per block (steady state) |
|---|---|---|---|
| RS(15,11) | 3 | 6 | 9.2 |
| RS(252,236) on the board | 4 | 63 | 69.3 |

For the reference profile at S = 1 and a 200 MHz clock: 1.6 Gbit/s coded on the wire, 1.5 Gbit/s of data. At S = 8 (a 64-bit bus, m = 8) the same clock gives 12.8 Gbit/s coded, which is where the 802.3-class profiles live.

## 17.2 What scales with S

*Table 5.2: Scaling with symbols per beat*

| Block | Scaling | Reason |
|---|---|---|
| encoder LFSR | S symbols advance per cycle | S single steps unrolled; the taps are constant, so it is one XOR network |
| syndrome cells | S-fold Horner step | S unrolled steps per cell, the state after the beat's count taken |
| Chien cells | S positions per cycle | one register set per cell, S lane constants alpha^(iu), the step constant alpha^(iS) |
| Forney | S corrections per cycle | S lane sums, S inverses and S multiplies |
| block and output FIFOs | width x S | depth in beats |
| key-equation solver | unchanged | it works on the 2t syndromes, not the stream |

The Forney row is the cost to watch: the cores as built take the S-inverse form, so a correction can land in every lane of a beat in the same cycle and the walk never stalls. The alternative – one inverse and a stall of up to t cycles per block on dense errors – is the trade to revisit if the characterisation shows the inverses dominating at large S.

# 18 Latency

All figures analytic at revision 0.1, one symbol per beat.

## 18.1 Encoder

*Table 5.3: Encoder latency*

| From | To | Cycles |
|---|---|---|
| first data symbol in | first data symbol out | pipeline only: about 3 (unpack, skid, pack) |
| k-th data symbol in | first parity symbol out | pipeline + 1 |
| k-th data symbol in | last parity symbol out | pipeline + 2t |

## 18.2 Decoder

*Table 5.4: Decoder latency*

| Phase | Cycles | Notes |
|---|---|---|
| receive the block | n | syndromes complete with the last symbol |
| key equation | 2t (riBM) or <= 2t (Euclid) | skipped when the syndromes are all zero |
| Chien + Forney + re-check walk | n | one position per cycle; the block is released only after its verdict |
| drain | k | the data symbols leave while the next block is walked |
| pipeline | about 8 | descriptor skids, solver handoff, the correct-stage register, the verdict stage, output FIFO |
| first symbol in to first symbol out | about 2n + 2t + 6 | 532 for RS(255,239) |
| **first symbol in to last symbol out** | **about 2n + 2t + k + 6** | 771 |

For a shortened code the n above is the shortened length (204 for DVB). For S symbols per beat, divide the n terms by S; the 2t solver term does not shrink.

## 18.3 Buffer sizing

The block FIFO must hold the current block while the next arrives during the Chien walk: depth = n + 2t + 8 symbols, rounded up to a power of two – 512 for the reference profile, 1024 for RS(544,514). The output FIFO holds a block's data symbols while it waits for its verdict and the next block's while that one is walked: depth = 2k + 8 rounded up, at 2m + 2 bits per entry (received symbol, correction, hit, last) – 512 x 18 for the reference profile. The release-on-verdict rule costs this second buffer and the extra n cycles of first-symbol latency; it is the price of never emitting an altered symbol on an uncorrectable block.

# 19 Resources

Analytic counts at revision 0.1, from `docs/rs_fub_catalog.md`; no synthesis has been run. Chapter 6.5 is the plan that replaces this table with measurements.

## 19.1 GF primitives per core, reference profile

*Table 5.5: GF primitive counts, RS(255,239), t = 8, m = 8, S = 1*

| Primitive | Encoder | Decoder (riBM) | Decoder (Euclid) | Formula |
|---|---|---|---|---|
| `gf_mul_const` | 16 | 33 | 33 | encoder 2t; decoder 2t + (t+1) + t |
| `gf_mul` | 0 | 52 | 66 | riBM 2(3t+1) + 2; Euclid 8t + 2; + t with erasures |
| `gf_inv` | 0 | 1 | 1 | one, in Forney |
| GF symbol registers | 16 | 75 | 89 | encoder 2t; decoder 2t + (t+1) + solver array |

## 19.2 Storage

*Table 5.6: Storage*

| Item | Size | Formula |
|---|---|---|
| block FIFO | 512 x 8 bits (one BRAM) | (n + 2t + 8, rounded up) x m (+1 erasure bit when built) |
| output FIFO | 512 x 18 bits (one BRAM) | (2k + 8, rounded up) x (2m + 2): received symbol, correction, hit, last |
| re-check syndromes | 2t cells | a second `syndrome_unit` over the corrected stream |
| log / antilog tables (`gf_inv`) | 2 x 256 x 8 bits | 2 x 2^m x m; LUT ROM at m = 8, BRAM at m = 10 |
| skid buffers | 2 (encoder), 3 (decoder), 2 deep | stage boundaries |

## 19.3 Order of magnitude

A `gf_mul_const` at m = 8 is at most 8 XOR trees of up to 8 inputs; a `gf_mul` is an 8 x 8 AND array with reduction, on the order of 60-80 LUT6 equivalents. That puts the reference decoder's arithmetic near 4-5 k LUTs on a 7-series part before control and the buffer, and the encoder near 200. The converters-area dwidth blocks and the bridge's monitors give the repository calibration points for what a block of this size does on Artix-7 and Kintex-7; the first out-of-context run will say where the RS decoder lands.

## 19.4 Scaling

- CCSDS RS(255,223), t = 16: every 2t-scaled row doubles (32 syndrome cells, 49 riBM elements, 17 Chien cells); plus the dual-basis conversion at both boundaries.
- 802.3 RS(544,514), m = 10: every multiplier grows from 8 x 8 to 10 x 10, the tables to 1024 entries, the buffer to 1024 deep; S > 1 is a requirement, not an option, at those rates.
- S symbols per beat: syndrome and Chien cells x S; solver unchanged.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 20 System Requirements

What a consumer must provide, and what it gets.

## 20.1 The consumer provides

- **Block boundaries.** `in_last` on the k-th data symbol (encoder) or n-th coded symbol (decoder) of every block. The consumer's scheduler, framer or job control already knows where blocks end; the core checks and reports, it does not infer.
- **Room for the rate change.** The encoder emits n symbols for every k it takes; the decoder emits k for every n. The consumer's downstream (encoder) or upstream (decoder) absorbs the difference, or provides a FIFO of 2t symbols so the encoder's parity drain does not stall its input.
- **A single clock** and the repository's active-low asynchronous reset.
- **Erasure knowledge**, if `ENABLE_ERASURES`: which symbols are known bad, per symbol, with the data.
- **Configuration at build time.** The profile is parameters; nothing about the code is set at run time in the bare core.

## 20.2 The consumer gets

- Systematic encoding: data pass through unchanged and in order, parity follows.
- A per-block verdict on the decoder: ok, corrected (with count), uncorrectable, framing error – valid with the last symbol of the block, so the consumer can act on the block it has just received.
- Data as received on an uncorrectable block, never silently altered.
- Determinism: no run-time state survives a block except the counters; the same input block yields the same output block every time.

## 20.3 Environment

- SystemVerilog-2012, synthesisable, no assertions in RTL (properties live in `formal/`), lint-clean under Verilator per the repository gate.
- Filelists under `rtl/filelists/`, registered in `bin/filelists.toml`, one per top, from the first module.
- Python golden model dependencies (`reedsolo`, `galois`) are DV-only.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 21 Parameter Configuration

Every parameter of the two cores and the standalone tops. **TBD** marks a PRD decision not yet taken; the value shown is the working default the reference profile uses.

## 21.1 Code

*Table 6.1: Code parameters*

| Parameter | Type | Range | Default | Meaning | PRD |
|---|---|---|---|---|---|
| SYMBOL_WIDTH | int | 3..16 | 8 | m; the field is GF(2^m) | D1 decided |
| T_SYMBOLS | int | 1..(2^m-1)/2 | 8 | correctable symbols per block; 2t parity | D2 decided |
| N_SYMBOLS | int | 2t+1 .. 2^m-1 | 2^m - 1 | block length; < 2^m-1 is a shortened code | D3 decided |
| PRIM_POLY | int | irreducible of degree m | 0x11D | primitive polynomial; selects the field representation | D8 per profile |
| FIRST_ROOT | int | 0 .. 2^m-2 | 0 **TBD** | b, first root of g(x); DVB 0, CCSDS 112 | D8 |
| DUAL_BASIS | bit | | 0 | CCSDS dual-basis symbol representation at the boundaries | D8 |
| ENABLE_ERASURES | bit | | 0 **TBD** | erasure input and erasure-locator logic | D5 |

| Parameter | Type | Range | Default | Meaning | PRD |
|---|---|---|---|---|---|
| | | | | (decoder) | |
| KES_ALGO | string | "RIBM", "EUCLID" | "RIBM" | key-equation solver | D11 decided |

Derived: K_SYMBOLS = N_SYMBOLS - 2*T_SYMBOLS; elaboration fails if it is below 1.

Every code parameter is fixed at elaboration (Sean, 2026-09-30). There is no run-time t or n: the arrays are generate loops sized by t, the generator polynomial is computed by gf_pkg when the design elaborates, and a block format change is a rebuild. A consumer that must mix formats in one instance would need a T_MAX sizing parameter and a cfg_t register, which is not in this specification.

## 21.2 Interface

*Table 6.2: Interface parameters*

| Parameter | Type | Range | Default | Meaning | PRD |
|---|---|---|---|---|---|
| `DATA_WIDTH` | int | multiple of `SYMBOL_WIDTH` | 8 | beat width; elaboration fails if not a multiple | D1 decided |
| `SYMBOLS_PER_BEAT` | derived | | 1 | `DATA_WIDTH / SYMBOL_WIDTH`, the throughput factor; tested at 1, 3, 4 and 8 | D6 decided |
| `INTAKE_IF` | string | `"NONE"`, `"AXIS"`, `"AXI4"` | `"NONE"` | intake adapter (standalone tops) | D9 decided |
| `OUTLET_IF` | string | `"NONE"`, `"AXIS"`, `"AXI4"` | `"NONE"` | outlet adapter | D9 decided |
| `ENABLE_SCRAMBLER` | bit | | 0 | line scrambler stage | D12 decided |
| `SCRAMBLER_POLY`, `SCRAMBLER_SEED`, `SCRAMBLER_AFTER_ENCODER` | int, int, bit | | DVB values | the standard's PRBS | D12 |

Adapter-specific parameters are in tables 4.5 and 4.7.

## 21.3 Open decisions in one place

*Table 6.3: Open PRD decisions and their defaults*

| Decision | What fixes it | Effect if left at the default |
|---|---|---|
| D4 encoder-only | a transmit-only or RAID-write consumer | both cores are built; a consumer instantiates what it needs |
| D5 erasures | storage / RAID consumer says yes | no erasure port, smaller decoder |
| D8 conventions | the standard | b = 0, no dual basis |
| D10 first consumer | you | none of the above changes until it appears |

# 22 Candidate Profiles

A profile is a resolved parameter set. These four come from the standards and papers in `References/`; the first consumer picks one or defines its own.

*Table 6.4: Candidate profiles*

| Profile | m | t | n / k | PRIM_POLY | FIRST_ROOT | Scrambler | Source | Notes |
|---|---|---|---|---|---|---|---|---|
| Reference (DVB outer code, full length) | 8 | 8 | 255 / 239 | $x^{8+x}4+x^{3+x}2+1$ | 0 | $x^{15+x}14+1$, seed 100101010000000, before the encoder | EN 300 429 / 744 sec. 4.3 (verified) | the profile every count in this document quotes |
| DVB shortened | 8 | 8 | 204 / 188 | same | 0 | same | same | 51 leading zero symbols dropped; MPEG-2 transport packets |
| CCSDS telemetry | 8 | 16 (or 8) | 255 / 223 (239) | $x^{8+x}7+x^2+x+1$ (CCSDS field) | 112 | $x^{17+x}14+1$, seed 11000111000111000, after | CCSDS 131.0-B-5 sec. 4 and 10 | dual-basis symbol representation; |

| Profile | m | t | n / k | PRIM_POLY | FIRST_ROOT | Scrambler | Source | Notes |
|---|---|---|---|---|---|---|---|---|
| | | | | | | the encoder (legacy: $x^{8+x}7+x^{5+x}3+1$, all ones) | | interleave depth 1-8 outside the core |
| Ethernet RS-FEC | 10 | 15 (KP4) / 7 (KR4) | 544 / 514, 528 / 514 | $x^{10+x}3+1$ | 0 | 802.3 scrambler is separate | IEEE 802.3 cl. 91 / 108 (cited) | S > 1 required; m = 10 tables |
| RAID erasure | 8 or 16 | any | any n <= 2^m-1 | any | – | none | Plank 1997 + 2003 | erasure-only; encoder + erasure decoder, no Chien search |

The DVB row is the one verified line by line against the standard's text (2026-09-29); CCSDS is read from the stored Blue Book; the 802.3 and RAID rows are from the cited documents and should be confirmed against them when chosen.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

# 23 Verification Strategy

## 23.1 Golden model

Every configuration is verified against a software Reed-Solomon implementation on the same blocks: `reedsolo` (MIT) with the profile's primitive polynomial, first root and shortening, cross-checked by `galois` (MIT) for the field tables the generator emits. The TB encodes with the model and checks the RTL encoder byte for byte; it injects errors into model-encoded blocks and checks the RTL decoder's corrected output and verdict against the model's decode.

## 23.2 Test matrix

*Table 6.5: Verification matrix*

| Axis | Values | Why |
|---|---|---|
| profile | each row of table 6.4 the build admits | field constants and shortening differ per profile |
| errors per block | 0, 1, t-1, t, t+1, random | 0 exercises the bypass; t+1 and more must match the reference decoder's verdict – uncorrectable, or the identical miscorrection where the code itself cannot tell – and an uncorrectable block's data must be exactly as received |
| reference model | `dv/tbclasses/rs_model.py` | the hardware algorithms in Python on reedsolo's field, validated against reedsolo's decoder; every decoder block is bit-exact against it |
| erasures (if built) | 0 .. 2t, mixed with errors up to 2e + f = 2t | the erasure budget |
| symbols per beat | 1, and the consumer's S | the S-fold datapaths |
| solver | riBM and Euclid | identical roots and values on every corrected block |
| scrambler | off and on | the OFF build must be tested as its own configuration |
| adapters | NONE/NONE first; then AXIS/AXIS and AXI4/AXI4; the mixed | the core is proven before any wrapper |

| Axis | Values | Why |
|---|---|---|
| | pairs follow | |
| back-pressure | none, random on both ends, sustained on the output | the valid/ready contract and the block buffer's full behaviour |
| framing | short block, long block, missing `last` | the framing-error path |

## 23.3 Test structure

cocotb tests under `dv/tests/` with TB classes in `dv/tbclasses/`, Pattern B (`cocotb_test_*` prefixes), the three run levels through both the REG_LEVEL grid and TEST_LEVEL depth, per the `test-patterns` skill. Traffic at the adapters uses the existing AXI-Stream and AXI4 BFMs; nothing drives a valid/ready port by hand. Registers are read by name through the generated regmap.

## 23.4 Formal

No assertions in RTL. Candidate properties for `formal/` blocks: the valid/ready contract at both ends of each core; the encoder's parity equals the golden LFSR for one block (bounded); the solver's degree invariant (deg Lambda <= number of iterations) and the equivalence of the two solvers on symbolic syndromes for small t.

## 23.5 Coverage

Functional coverage on the matrix above plus: every syndrome cell non-zero at least once, every Chien position rooted at least once across the run, Forney with a zero and a maximal error value, the uncorrectable path taken by both the degree check and the root-count check.

# 24 Synthesis and Implementation

## 24.1 Characterisation plan

Numbers in chapter 5 are analytic. They are replaced by measurements the way the repository measures every other block: an out-of-context synthesis fixture per configuration, on the same two parts the monitor and bridge characterisations use.

*Table 6.6: Characterisation steps*

| Step | Fixture | Output |
|---|---|---|
| 1 | `rs_encoder_core` and `rs_decoder_core`, reference profile, S = 1, Artix-7 100T -1 at 10 ns and Kintex-7 325T -2 at 6.667 ns | LUTs, flops, BRAM, WNS; the calibration for table 5.5 |
| 2 | decoder with `KES_ALGO = "EUCLID"` | the solver comparison in table 3.1 measured |
| 3 | S = 2, 4, 8 | the scaling in table 5.2 measured; the Forney stall design decided |
| 4 | CCSDS and 802.3 profiles | t and m scaling |
| 5 | standalone tops with each adapter pair | adapter cost; the monitors' cost is already known |

The fixture follows `projects/components/fabric-gen-ip/bridge/fpga/` (`make synth BRIDGE=<fixture>` on `make/fpga_flow.mk`) and reports through the same `summary.csv` shape, so the numbers land beside the bridge's and the monitor's.

## 24.2 Implementation notes

- All arithmetic is combinational XOR/AND networks; there are no carries, so the multiplier trees of `rtl/math` do not apply and the synthesiser's DSP inference is irrelevant.
- `gf_inv` at m = 8 is two 256-entry ROMs and infers to LUTs; at m = 10 it becomes BRAM or Itoh-Tsujii.
- The block buffer infers one BRAM at the reference depth; `REGISTERED = 1` on the FIFO adds a cycle and removes the read mux from the corrector's path if timing needs it.
- The riBM array has no cross-array feedback, so its cells may be pipelined per iteration if the clock demands; the Euclid array's degree compare cannot be pipelined without the degree-computationless variant.
- No clock gating in the core; boundary gating comes from the `_cg` wrapper variants at the adapters.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 25 Fields and Polynomials

## 25.1 Why error correction needs a finite field

Every arithmetic step inside the Reed-Solomon codec uses a finite field. The reason is not abstract: correction has to divide, and ordinary integer division does not behave well inside a hardware pipeline. A field is a number system where addition, subtraction, multiplication, and division (except by zero) all stay inside the same set and obey the rules we expect. If the set is finite, every operation has a fixed width, every result fits in an m-bit register, and there are no rounding or overflow surprises. That is why the symbols, the coefficients, the syndromes, and the error magnitudes all live in $GF\left(2^m\right)$.

## 25.2 What a field is

A field has four operations, each closed inside the set, with identities and inverses:

- addition has an identity $0$ and every element has a negative;
- multiplication has an identity $1$ and every nonzero element has a reciprocal;
- multiplication distributes over addition.

The rational numbers are a field, but they are infinite. Integers modulo *n* are finite, yet they are a field only when *n* is prime. Take $n=6$: the element $2$ has no multiplicative inverse because no integer $x$ satisfies $2x \equiv 1 \left( mod\, 6 \right)$. Worse, $2 \cdot 3 = 0$ in this system, so zero can be factored out of nonzero elements. That breaks division. When *n* is prime, every nonzero residue has an inverse, and the system is the field $GF\left(p\right)$.

For hardware we use characteristic 2: the field has $2^m$ elements, written $GF\left(2^m\right)$. The smallest such field is $GF\left(2\right)$, the two-element field $\left\{0,1\right\}$.

## 25.3 GF(2): the two-element field

In $GF(2)$, addition and subtraction are the same operation: exclusive OR.

*Table 7.1: Operations in GF(2)*

| a | b | a + b | a * b |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

Multiplication is the same as logical AND. The only nonzero element is $1$, and it is its own inverse. This is the arithmetic that every digital gate already does.

## 25.4 Polynomials over GF(2) and irreducibility

A polynomial over $GF(2)$ has coefficients that are only $0$ or $1$, and its arithmetic follows the table above. For example,

$$p(x) = x^4 + x + 1$$

has coefficients $1, 0, 0, 1, 1$. In binary this is `10011`. We add two such polynomials by XORing their coefficient vectors; there are no carries.

A polynomial is irreducible if it cannot be factored into lower-degree polynomials over the same field. The polynomial $x^4 + x + 1$ is irreducible over $GF(2)$, while $x^4 + x^2 + 1$ factors as $(x^2 + x + 1)^2$ and is therefore not. Irreducibility is the analogue of primeness for integers: it is what lets us build a field by taking polynomials modulo $p(x)$.

## 25.5 Building GF(2^m)

To build $GF(2^4)$, we take all polynomials over $GF(2)$ of degree less than 4, modulo an irreducible degree-4 polynomial. There are $2^4 = 16$ such polynomials, one for every 4-bit pattern. Addition is XOR. Multiplication is ordinary polynomial multiplication followed by reduction modulo $p(x)$, which just means replacing every $x^4$ that appears with $x+1$ (because $x^4 \equiv x+1 \pmod{x^4+x+1}$) until the degree drops below 4.

The residue of $x$ is called the primitive element $\alpha$. Because $p(x)$ is primitive, the powers of $\alpha$ generate every nonzero field element before they repeat. That gives us a compact multiplicative description of the whole field: every nonzero element is $\alpha^i$ for some $i$, and $\alpha^{15} = 1$.

## 25.6 The GF(2^4) log/antilog table

The following table is the reference for every worked example in this chapter. It uses $p(x) = x^4 + x + 1$ (`10011`). Each row shows one power of $\alpha$ as both a 4-bit vector and a polynomial in $\alpha$.

*Table 7.2: Powers of alpha in GF(2^4) with p(x) = x^4 + x + 1*

| i | alpha^i (binary) | alpha^i (polynomial) |
|---|---|---|
| 0 | 0001 | 1 |
| 1 | 0010 | alpha |
| 2 | 0100 | alpha^2 |
| 3 | 1000 | alpha^3 |
| 4 | 0011 | alpha + 1 |
| 5 | 0110 | alpha^2 + alpha |
| 6 | 1100 | alpha^3 + alpha^2 |
| 7 | 1011 | alpha^3 + alpha + 1 |
| 8 | 0101 | alpha^2 + 1 |
| 9 | 1010 | alpha^3 + alpha |
| 10 | 0111 | alpha^2 + alpha + 1 |
| 11 | 1110 | alpha^3 + alpha^2 + alpha |
| 12 | 1111 | alpha^3 + alpha^2 + alpha + 1 |
| 13 | 1101 | alpha^3 + alpha^2 + 1 |
| 14 | 1001 | alpha^3 + 1 |

To multiply two nonzero elements, add their exponents modulo 15. To divide, subtract exponents modulo 15. The zero element has no log; anything multiplied by 0 is 0.

## 25.7 Worked multiplies in GF(2^4)

We will use the table in two ways: by adding exponents, and by expanding vectors when an intermediate step needs to be visible.

*Table 7.3: Sample multiplications in GF(2^4)*

| Product | Exponent form | Vector form | Result |
|---|---|---|---|
| alpha^4 * alpha^8 | alpha^12 | 0011 * 0101 | 1111 = alpha^12 |
| alpha^10 * alpha^7 | alpha^17 = alpha^2 | 0111 * 1011 | 0100 = alpha^2 |
| alpha^5 * alpha^13 | alpha^18 = alpha^3 | 0110 * 1101 | 1000 = alpha^3 |

The second column shows the rule: add exponents and reduce modulo 15. The third column shows the same operation as vector XOR products, with each partial product shifted and reduced by `10011`. Both views are the same operation; the exponent view is faster by hand, and the vector view is what the gates implement.

## 25.8 A worked reduction

It is worth seeing once how the vector view produces the same answer as the exponent view. Multiply $\alpha^4$ (`0011`) by $\alpha^8$ (`0101`). As polynomials these are $(\alpha+1)$ and $(\alpha^2+1)$. Their product is

$$(\alpha+1)(\alpha^2+1)=\alpha^3+\alpha+\alpha^2+1.$$

In binary this is `0011 * 0101`. Treat the multiplier `0101` as bits $b_0=1$, $b_2=1$. The partial products are the multiplicand shifted by those positions: `0011` and `1100`. Adding them with XOR gives `1111`. The degree is already below 4, so no reduction is needed, and the result is $\alpha^{12}$. That matches $\alpha^4\cdot\alpha^8=\alpha^{12}$.

Now try $\alpha^5$ (`0110`) times $\alpha^{13}$ (`1101`). The unreduced product is degree 6, so a reduction appears:

$$\alpha^5\cdot\alpha^{13}=\alpha^{18}=\alpha^{18-15}=\alpha^3.$$

In vectors: `0110 * 1101`. The multiplier bits are $b_0=1$, $b_2=1$, $b_3=1$, giving partial products `0110`, `110000`, and `1100000`. Reduce each power of $x^4$ or higher using $x^4=x+1$. The term $x^5=x\cdot x^4$ becomes $x^2+x$, and $x^6=x^2\cdot x^4$ becomes $x^3+x^2$. Collecting all terms and XORing coefficients gives `1000`, which is $\alpha^3$. A hardware multiplier performs exactly this reduction, but in parallel rather than term by term.

## 25.9 The symbol view

In this codec a symbol is one field element. With $m = 8$ a symbol is a byte; with $m = 4$ it is a nibble; with $m = 10$ it is the 10-bit symbol used by the Ethernet RS-FEC profile from chapter 6.3. The RS block is a sequence of symbols, not bits. When two symbols are multiplied, they are multiplied as field elements using the rules above, not as binary integers. This is the first thing that confuses newcomers: `0x02 * 0x03` in GF(256) is not `0x06`. It is whatever the field polynomial says it is, and there are no carries anywhere in the codec.

The chapter 1.3 definitions call this out explicitly: all arithmetic lives in $GF\left(2^m\right)$. Addition is XOR. Subtraction is also XOR, because in characteristic 2 every element is its own negative. That single fact is why the generator polynomial roots are written $\left(x - \alpha^j\right)$ and $\left(x + \alpha^j\right)$ interchangeably, and why correction later will add the computed error value to the received symbol instead of subtracting it.

The choice of $m$ is the parameter `SYMBOL_WIDTH` from chapter 6.2. The choice of primitive polynomial is `PRIM_POLY`. Together they fix the exact log/antilog table the hardware uses. Change the polynomial and the binary representation of every nonzero symbol changes, even though the abstract code does not.

## 25.10 Hardware tie: a register is a field element

An m-bit register holding one symbol is exactly one element of $GF(2^m)$. Addition is a row of XOR gates. Multiplication by a constant is a fixed XOR network: each output bit is the XOR of some subset of the input bits, chosen once from the vector form of the constant. That is why the encoder taps, syndrome roots, Chien stepping constants, and Forney denominator are all cheap in silicon: they are multiplications by known constants, not general field multipliers.

A general field multiplier (`gf_mul`) is still just ANDs and XORs, with one variable array feeding a network of constant arrays. There is no carry chain, so the delay is logarithmic in the number of terms, not bit-serial. A parallel multiplier is larger than a carry-save integer multiplier of the same width, but it is also faster and deterministic, which is why the decoder can afford one multiplier per syndrome or Chien lane and still meet timing.

The GF layer described in chapter 3.1 and the FUB catalog is where these operations live. Every module that needs field arithmetic instantiates one of a small family of blocks: `gf_mul` (general multiply), `gf_mul_const` (multiply by a fixed constant, which synthesis collapses into pure XOR trees), `gf_inv`, and `gf_syndrome_cell` built on constant multiplies. Addition never needs a module; it is just XOR. When you see `gf_pkg` referenced in the RTL, it is the package of constant functions that implements the field arithmetic — multiply, inverse, powers, logs — from `SYMBOL_WIDTH` and `PRIM_POLY` at elaboration.

*Table 7.4: Field operations and their hardware forms*

| Math object | Hardware representation |
|---|---|
| symbol | m-bit register |
| a + b | m XOR gates |
| a * b (general) | m-by-m AND/XOR array |
| a * constant | fixed XOR network |
| alpha^i | lookup or constant register |

With the field in place, section 7.2 turns symbols into a code.

# 26 From Field to Code

## 26.1 Codewords as polynomials over GF(2^m)

A Reed-Solomon block is a polynomial whose coefficients are field elements. If the block has $n$ symbols, we write

$$c(x) = c_0 + c_1 x + \ldots + c_{n-1} x^{n-1},$$

where each $c_i$ is one element of $GF(2^m)$. This is different from the bit-polynomial view used in CRCs or binary BCH. There the coefficients are single bits; here the coefficients are whole symbols. A single coefficient can already be any of $2^m$ values, so the polynomial carries much more information per term. A CRC appends the same number of check symbols but only detects: it can flag any burst of up to $2t$ bits, yet it cannot say where the error is. An RS code of the same parity size can correct $t$ symbol errors anywhere in the block, and each symbol is $m$ bits, so a single error locator can cover an entire burst.

Of the $n$ coefficients, $k$ are data symbols and $2t$ are parity symbols, with $n = k + 2t$. The code is called RS($n$, $k$) over $GF(2^m)$. The parameter `T_SYMBOLS` from chapter 6.2 is this $t$, and `N_SYMBOLS` is $n$. A codeword is valid exactly when it is divisible by $g(x)$, which is the same as saying it vanishes at every root of $g(x)$.

## 26.2 The generator polynomial

Reed-Solomon is the $q$-ary cousin of BCH. Its generator polynomial has $2t$ consecutive powers of $\alpha$ as roots:

$$g(x) = \prod_{i=0}^{2t-1} \left(x - \alpha^{b+i}\right).$$

In characteristic 2, subtraction is the same as addition, so $\left(x - \alpha^{j}\right)$ and $\left(x + \alpha^{j}\right)$ are identical. The value $b = 1$ is called narrow-sense; chapter 6.2 exposes $b$ as the `FIRST_ROOT` parameter, and profiles move it (DVB uses $b = 0$, CCSDS $b = 112$).

Unlike binary BCH, RS does not need minimal polynomials. In binary BCH the coefficients must be bits, so each root $\alpha^{j}$ has to be grouped with its conjugates under squaring to force binary coefficients. RS already allows coefficients to be any field element, so the simple product above has coefficients in $GF\left(2^{m}\right)$ directly. The degree of $g(x)$ is exactly $2t$, which equals $n - k$. Each root contributes one degree to $g(x)$, and therefore one parity symbol to the block. That direct count is why RS($n$, $k$) always has exactly $n - k = 2t$ parity symbols.

## 26.3 Why RS is maximum-distance-separable

A code's minimum distance $d_{min}$ is the smallest number of symbol positions in which any two distinct codewords differ. The Singleton bound says $d_{min} \le n - k + 1$. Reed-Solomon meets this bound exactly:

$$d_{min} = n - k + 1 = 2t + 1.$$

That is why RS can correct any $t$ symbol errors: $2t + 1$ guarantees that any two codewords are far enough apart that a sphere of radius $t$ around each one does not overlap any other. Decoding is unambiguous: the decoder always finds the single closest codeword, or declares failure if too many symbols are wrong.

It also explains why RS is the default choice for burst channels such as flash pages and optical links, which are exactly the use cases in chapter 2.1. A burst that corrupts several consecutive bits usually lands inside one or two symbols; RS counts symbol errors, not bit errors, so a 10-bit burst inside one symbol costs the same as a single flipped bit inside another symbol. The MDS property tells us the code is using its parity as efficiently as any linear code possibly can.

## 26.4 Shortened codes

A full-length RS code has $n = 2^m - 1$. Practical standards often use a shorter block by treating the leading $2^m - 1 - n$ symbols as zero and not transmitting them. The DVB RS(204,188) profile in chapter 6.3 is built this way from RS(255,239) over $GF(2^8)$: 51 leading zero symbols are dropped, leaving a 204-symbol block that is mathematically the tail of the full codeword.

Shortening does not change the generator polynomial, the field, or the number of correctable errors. The only hardware changes are in bookkeeping. The syndrome cells treat the missing leading positions as zero, so their accumulators start at zero and remain zero for those positions. The Chien search offsets its position counter so that it still walks the 204 transmitted positions rather than all 255. The decoder output drops the implicit zeros, so the consumer sees exactly the shortened packet.

## 26.5 Worked construction of RS(15,11)

We now build a concrete code: RS(15,11) with $t=2$ over $GF(2^4)$, using the field from section 7.1. The generator has roots $\alpha$, $\alpha^2$, $\alpha^3$, $\alpha^4$:

$$g(x)=(x+\alpha)(x+\alpha^2)(x+\alpha^3)(x+\alpha^4).$$

We multiply the factors pairwise. First pair $(x+\alpha)(x+\alpha^2)$:

$$(x+\alpha)(x+\alpha^2)=x^2+(\alpha+\alpha^2)x+\alpha^3=x^2+\alpha^5 x+\alpha^3.$$

Second pair $(x+\alpha^3)(x+\alpha^4)$:

$$(x+\alpha^3)(x+\alpha^4)=x^2+(\alpha^3+\alpha^4)x+\alpha^7=x^2+\alpha^7 x+\alpha^7.$$

Now multiply the two quadratics:

$$g(x)=(x^2+\alpha^5 x+\alpha^3)(x^2+\alpha^7 x+\alpha^7).$$

Collecting terms gives

$$g(x)=x^4+(\alpha^5+\alpha^7)x^3+(\alpha^3+\alpha^{12}+\alpha^7)x^2+(\alpha^{12}+\alpha^{10})x+\alpha^{10}.$$

Using Table 7.2 to add the vector forms, the coefficients simplify to:

*Table 7.5: Generator polynomial g(x) for RS(15,11)*

| Coefficient | Value | Vector |
|---|---|---|
| g_0 | alpha^10 | 0111 |
| g_1 | alpha^3 | 1000 |
| g_2 | alpha^6 | 1100 |
| g_3 | alpha^13 | 1101 |
| g_4 | 1 | 0001 |

So

$$g(x)=x^4+\alpha^{13}x^3+\alpha^6 x^2+\alpha^3 x+\alpha^{10}.$$

Every codeword polynomial is a multiple of this $g(x)$. That is the entire definition of the code. The four roots guarantee that any valid codeword $c(x)$ satisfies $c(\alpha)=c(\alpha^2)=c(\alpha^3)=c(\alpha^4)=0$. Those four evaluations are exactly what the syndrome cells in chapter 3.2 compute later, only on the received word instead of a known codeword.

## 26.6 Systematic encoding

The encoder in chapter 3.2 produces codewords in systematic form: the $k$ data symbols pass through unchanged, followed by the $2t$ parity symbols. Mathematically,

$$c(x) = x^{2t} d(x) + r(x),$$

where $d(x)$ is the data polynomial and

$$r(x) = x^{2t} d(x) \bmod g(x).$$

The parity $r(x)$ is the remainder after polynomial long division of $x^{2t} d(x)$ by $g(x)$. The hardware LFSR is that long division running in real time: the $2t$ registers hold the current remainder, and each incoming data symbol updates them by one degree.

Here is how one LFSR step works. Let the current remainder be $r(x) = r_0 + r_1 x + r_2 x^2 + r_3 x^3$. When the next data symbol $s$ arrives, we conceptually append it to the high-degree end, forming $x \cdot r(x) + s\,x^4$. Since $g(x)$ is monic, $x^4 \equiv g_3 x^3 + g_2 x^2 + g_1 x + g_0 \pmod{g(x)}$. So the new remainder is

$$x \cdot r(x) + s \cdot \left(g_3 x^3 + g_2 x^2 + g_1 x + g_0\right),$$

with all arithmetic in $GF(2^m)$. The hardware implements this with XOR gates and constant multipliers tied to the generator coefficients. After the last data symbol the registers contain the parity; the mux then switches from data to parity and drains the registers out.

## 26.7 Worked systematic encode

Take the following 11 data symbols as $d_0, d_1, \ldots, d_{10}$:

*Table 7.6: Data symbols for the RS(15,11) worked example*

| Index | d_i |
|---|---|
| 0 | 0 |
| 1 | 1 |
| 2 | alpha |
| 3 | alpha^4 |
| 4 | alpha^2 |
| 5 | alpha^8 |
| 6 | alpha^5 |
| 7 | alpha^10 |
| 8 | alpha^3 |
| 9 | alpha^14 |
| 10 | alpha^9 |

The encoder streams these in reverse polynomial order: $d_{10}, d_9, \ldots, d_0$. After each symbol the LFSR state is the current remainder. The table below shows the high-to-low remainder registers $r_3, r_2, r_1, r_0$ after each input. All operations are in $GF\left(2^4\right)$.

*Table 7.7: LFSR remainder evolution during encoding*

| Step | Input | r3 | r2 | r1 | r0 |
|---|---|---|---|---|---|
| 0 | alpha^9 | alpha^7 | alpha^0 | alpha^12 | alpha^4 |
| 1 | alpha^14 | alpha^3 | alpha^2 | 0 | alpha^11 |
| 2 | alpha^3 | alpha^2 | 0 | alpha^11 | 0 |
| 3 | alpha^10 | alpha^2 | alpha^14 | alpha^7 | alpha^14 |
| 4 | alpha^5 | 0 | 0 | alpha^9 | alpha^11 |
| 5 | alpha^8 | alpha^6 | alpha^4 | 0 | alpha^3 |
| 6 | alpha^2 | alpha^0 | alpha^9 | alpha^2 | alpha^13 |
| 7 | alpha^4 | alpha^4 | alpha^12 | alpha^11 | alpha^11 |
| 8 | alpha^1 | alpha^1 | alpha^1 | alpha^5 | alpha^10 |
| 9 | alpha^0 | alpha^5 | alpha^0 | alpha^6 | alpha^14 |

| Step | Input | r3 | r2 | r1 | r0 |
|---|---|---|---|---|---|
| 10 | 0 | alpha^14 | alpha^1 | alpha^6 | alpha^0 |

The final remainder is the parity, from low degree to high:

$$r_0 = \alpha^0, r_1 = \alpha^6, r_2 = \alpha^1, r_3 = \alpha^{14}.$$

In stream order the encoder sends data first, then parity, because chapter 3.2 describes a systematic encoder that passes the data beats through and appends the parity at the end. The polynomial coefficient order $c_0, c_1, \ldots, c_{14}$ is the opposite: the parity coefficients sit at the low-degree end and the data coefficients at the high-degree end. The table below lists the codeword in polynomial order so that $c(x) = c_0 + c_1 x + \ldots + c_{14} x^{14}$ is easy to read. Position $j$ is the coefficient index, so position 0 is the $x^0$ coefficient and the encoder sends position 14 first.

*Table 7.8: Final RS(15,11) codeword*

| Position | c_i |
|---|---|
| 0 | alpha^0 |
| 1 | alpha^6 |
| 2 | alpha^1 |
| 3 | alpha^14 |
| 4 | 0 |
| 5 | 1 |
| 6 | alpha |
| 7 | alpha^4 |
| 8 | alpha^2 |
| 9 | alpha^8 |
| 10 | alpha^5 |
| 11 | alpha^10 |
| 12 | alpha^3 |
| 13 | alpha^14 |
| 14 | alpha^9 |

This exact codeword is used again in section 7.3, where we inject errors and correct them. As a sanity check, dividing the full codeword polynomial by $g(x)$ leaves remainder zero, which is the defining property of any RS codeword. If the remainder were nonzero, either the generator or the encoding table would contain an arithmetic error, and section 7.3 could not possibly decode correctly.

# 27 Syndromes, the Key Equation, and Correction

## 27.1 The received polynomial

The channel adds an error polynomial to the codeword:

$$R(x) = c(x) + e(x).$$

Each coefficient of $e(x)$ is a field element. A nonzero coefficient $e_j$ means the polynomial coefficient $c_j$ — and therefore the symbol the RTL transmits at position $n - 1 - j$ — is wrong, and the value can be any nonzero field element. This is the fundamental difference from binary BCH, where every error value is implicitly $1$. Reed-Solomon correction must find both the positions and the values.

Because an error value can be any of $2^m - 1$ nonzero symbols, an RS decoder does more work than a binary decoder. Binary BCH only needs to know where the errors are; RS also needs to know by how much. That is why the decoder has both a locator polynomial and an evaluator polynomial, and why the final stage is a Forney computation rather than a simple bit flip.

## 27.2 Syndromes

A received word is a valid codeword if and only if it has every root of $g(x)$ as a root. We define the syndromes as the received polynomial evaluated at those roots. For narrow-sense $b=1$:

$$S_i = R(\alpha^i), i = 1, 2, \ldots, 2t.$$

Equivalently, $S_i = R(\alpha^{b+i-1})$ for $i = 1 \ldots 2t$, matching the definition in chapter 1.3. All $2t$ syndromes are used: the evenness shortcut of binary BCH does not apply, because for symbol errors the error values are arbitrary field elements and $e^2 \neq e$ in general, so $S_{2j} \neq S_j^2$. If every syndrome is zero, the block is a codeword and the decoder passes it through unchanged. If any syndrome is nonzero, correction begins.

The syndrome cells in chapter 3.2 compute each $S_i$ by Horner evaluation as the block streams in:

$$S_i \leftarrow \alpha^i \cdot S_i + R_j.$$

Here $R_j$ is the symbol that just arrived, where arrival 0 is the first-transmitted (highest-degree) coefficient. One multiply-accumulate per syndrome, per symbol. The constant $\alpha^i$ is a known field element, so the multiplier is a fixed XOR network rather than a general field multiplier. With $S$ symbols per beat, the syndrome cell performs $S$ such updates in parallel, each at a different power of $\alpha$ corresponding to the symbol's position within the beat.

## 27.3 The key equation

Let the syndrome polynomial be

$$S(x) = S_1 + S_2 x + S_3 x^2 + \ldots + S_{2t} x^{2t-1}.$$

The error locator polynomial is

$$\Lambda(x) = \prod_{k=1}^{v} (1 - X_k x),$$

where $X_k = \alpha^{j_k}$ is the locator for an error at position $j_k$, and $v$ is the number of errors. The error evaluator polynomial is $\Omega(x)$. They satisfy the key equation:

$$\Lambda(x) S(x) \equiv \Omega(x) (mod\, x^{2t}).$$

The solver inverts this equation: given the syndromes, it finds $\Lambda(x)$ and $\Omega(x)$. The default implementation in this codec is riBM, described in chapter 3.3; the alternative is the inversionless Euclidean algorithm. The math here is the same for both. We only need to know that the solver outputs $\Lambda(x)$ and $\Omega(x)$.

The locator polynomial carries the positions: its roots are the reciprocals of the error locators. The evaluator polynomial carries the magnitudes: combined with the derivative of $\Lambda$, it yields each error value. The key equation ties them together through the syndromes, which are the only thing the decoder can measure directly.

## 27.4 Chien search

Once $\Lambda(x)$ is known, the decoder finds its roots. A root at $x = \alpha^{-j}$ means an error at position $j$:

$$\Lambda(\alpha^{-j}) = 0 \Leftrightarrow \text{position } j \text{ is in error.}$$

The Chien search evaluates $\Lambda(x)$ at $\alpha^{0}, \alpha^{-1}, \ldots, \alpha^{-(n-1)}$, one position per cycle, while the block buffer reads out the same positions. This is the root-finding step described in chapter 3.2.

The evaluation is done incrementally. If the current value is $\Lambda(\alpha^{-j})$, then the next value is $\Lambda(\alpha^{-(j+1)}) = \Lambda(\alpha^{-j} \cdot \alpha^{-1})$, which is obtained by multiplying each register by the appropriate constant. That constant multiply is again a fixed XOR network. A root is detected when the evaluated value is exactly zero, which is just an *m*-bit zero test.

## 27.5 The Forney formula

For each located root $X_k^{-1} = \alpha^{-j_k}$, the error value is

$$e_{j_k} = -\frac{\Omega\left(X_k^{-1}\right)}{\Lambda'\left(X_k^{-1}\right)}.$$

The formal derivative $\Lambda'(x)$ drops the even-powered terms and halves the exponent of each odd-powered term. In characteristic 2, halving an odd exponent is just reduction modulo 2, so

$$\frac{d}{dx} x^{2i+1} = x^{2i}, \frac{d}{dx} x^{2i} = 0.$$

For our $t = 2$ example, $\Lambda(x) = 1 + \Lambda_1 x + \Lambda_2 x^2$, so $\Lambda'(x) = \Lambda_1$ is a single constant. The minus sign also vanishes in characteristic 2 because $-1 = 1$, so the formula becomes

$$e_{j_k} = \frac{\Omega\left(\alpha^{-j_k}\right)}{\Lambda'\left(\alpha^{-j_k}\right)}.$$

The evaluator $\Omega(x)$ encodes the error magnitudes because it absorbs the syndrome information that depends on how wrong each symbol is; the derivative $\Lambda'(x)$ removes the weight of the located root, leaving the bare error value. If $\Lambda'\left(X_k^{-1}\right) = 0$ while $\Omega\left(X_k^{-1}\right) \neq 0$, the Forney formula divides by zero and the decoder cannot resolve the magnitude. That is the “derivative was zero at a root” verdict in chapter 3.2.

## 27.6 Erasure math

When ENABLE_ERASURES is set, the decoder accepts symbols flagged as known bad. Each erasure costs one parity symbol instead of two. If $\nu$ erasures and $\mu$ errors are present, the code corrects them when

$$2\mu + \nu \le 2t\,.$$

The erasure port is described in chapter 4.1. As of chapter 6.2 the default parameter value is ENABLE_ERASURES = 0, so the port is absent and the erasure-locator logic is not generated unless a consumer enables it. The math is worth knowing anyway: erasures turn known-bad positions into extra correctable errors. A RAID controller that knows which drive failed, or a link layer that marks a dropped packet, can recover twice as many known failures as unknown ones with the same parity budget.

## 27.7 Worked decode of RS(15,11)

We take the codeword from section 7.2 and inject exactly two symbol errors:

- position $j=3$: add $\alpha^5$;
- position $j=11$: add $\alpha^9$.

In this example, $j$ is the polynomial coefficient index: the table lists $c_0, c_1, \ldots, c_{14}$ from low degree to high degree. The RTL transmits the high-degree end first, so the syndrome cell sees $c_{14}$ as arrival 0, $c_{13}$ as arrival 1, and so on down to $c_0$. The syndrome values below are computed from the polynomial coefficient order shown; if you stream the symbols through the hardware, reverse the table order.

The received word in polynomial coefficient order is:

*Table 7.9: Received word with two injected errors*

| Position | Received |
|---|---|
| 0 | alpha^0 |
| 1 | alpha^6 |
| 2 | alpha^1 |
| 3 | alpha^12 |
| 4 | 0 |
| 5 | 1 |
| 6 | alpha |
| 7 | alpha^4 |
| 8 | alpha^2 |
| 9 | alpha^8 |
| 10 | alpha^5 |
| 11 | alpha^13 |
| 12 | alpha^3 |
| 13 | alpha^14 |
| 14 | alpha^9 |

### 27.7.1 Computing the syndromes

Evaluating $R(x)$ at $\alpha, \alpha^2, \alpha^3, \alpha^4$ gives:

*Table 7.10: Syndromes of the received word*

| Syndrome | Value | Vector |
|---|---|---|
| S_1 | alpha^4 | 0011 |
| S_2 | alpha^6 | 1100 |
| S_3 | alpha^5 | 0110 |
| S_4 | alpha^0 | 0001 |

All four are nonzero, so the block is not a codeword. Notice that each syndrome is a field element, not a single bit, because the error values are field elements. The syndrome pattern encodes both where the errors are and how large they are.

### 27.7.2 Peterson solve for t = 2

For two errors we can solve directly. Let $\Lambda(x) = 1 + \Lambda_1 x + \Lambda_2 x^2$. The Newton identities are

$$\Lambda_1 S_2 + \Lambda_2 S_1 = S_3,$$

$$\Lambda_1 S_3 + \Lambda_2 S_2 = S_4.$$

In matrix form,

$$\begin{pmatrix} S_2 & S_1 \\ S_3 & S_2 \end{pmatrix} \begin{pmatrix} \Lambda_1 \\ \Lambda_2 \end{pmatrix} = \begin{pmatrix} S_3 \\ S_4 \end{pmatrix}.$$

The determinant is $S_2^2 + S_1 S_3 = \alpha^8$, whose inverse is $\alpha^7$. Solving gives

*Table 7.11: Error locator polynomial coefficients*

| Coefficient | Value | Vector |
|---|---|---|
| Lambda_1 | alpha^5 | 0110 |
| Lambda_2 | alpha^14 | 1001 |

So

$$\Lambda(x) = 1 + \alpha^5 x + \alpha^{14} x^2.$$

Any nonzero scalar multiple of this locator has the same roots, so the riBM solver in section 7.5 may output a scaled version. The Forney stage there carries the same scale, and its `off = 2t` factor absorbs it, so the recovered error values stay identical.

### 27.7.3 Chien search in the example

We evaluate $\Lambda(\alpha^{-j})$ for $j = 0, 1, \ldots, 14$, still using the polynomial coefficient index from the table above. The RTL labels the transmitted position of $c_j$ as $n - 1 - j$, but the set of roots is the same.

*Table 7.12: Chien search over the 15 positions*

| j | alpha^{-j} | Lambda(alpha^{-j}) |
|---|---|---|
| 0 | alpha^0 | alpha^11 |
| 1 | alpha^14 | alpha^13 |
| 2 | alpha^13 | alpha^11 |
| 3 | alpha^12 | 0 |
| 4 | alpha^11 | alpha^12 |
| 5 | alpha^10 | alpha^4 |
| 6 | alpha^9 | alpha^6 |
| 7 | alpha^8 | alpha^13 |
| 8 | alpha^7 | alpha^4 |
| 9 | alpha^6 | alpha^0 |
| 10 | alpha^5 | alpha^6 |
| 11 | alpha^4 | 0 |
| 12 | alpha^3 | alpha^1 |
| 13 | alpha^2 | alpha^1 |
| 14 | alpha^1 | alpha^12 |

The zeros occur at $j=3$ and $j=11$, exactly the injected error positions. The degree of $\Lambda(x)$ is 2, matching the root count. If the search had found fewer roots than the degree — one root for a degree-2 locator, say — the error locators would not be distinct, and the block would be uncorrectable because the math does not describe a consistent error pattern.

### 27.7.4 Error evaluator and Forney

The syndrome polynomial is

$$S(x)=\alpha^4+\alpha^6 x+\alpha^5 x^2+x^3.$$

Multiplying by $\Lambda(x)$ and reducing modulo $x^4$ gives

$$\Omega(x)=\alpha^4+\alpha^5 x.$$

The formal derivative is $\Lambda'(x)=\alpha^5$. Applying the Forney formula at each root:

*Table 7.13: Forney error magnitudes*

| Position j | Omega(alpha^{-j}) | Lambda'(alpha^{-j}) | e_j |
|---|---|---|---|
| 3 | alpha^10 | alpha^5 | alpha^5 |
| 11 | alpha^14 | alpha^5 | alpha^9 |

The computed magnitudes match the injected errors. At $j=3$ the received symbol is $\alpha^{12}$ and the computed error is $\alpha^{5}$; adding them gives $\alpha^{12}+\alpha^{5}=\alpha^{14}$, which is the original $c_3$. At $j=11$ the received symbol is $\alpha^{13}$ and the computed error is $\alpha^{9}$; adding them gives $\alpha^{13}+\alpha^{9}=\alpha^{10}$, which is the original $c_{11}$. Adding them back (which is XOR in characteristic 2) restores the original codeword.

### 27.7.5 Verification

After correction, the four syndromes are all zero:

*Table 7.14: Post-correction syndromes*

| Syndrome | Corrected value |
|---|---|
| S_1 | 0 |
| S_2 | 0 |
| S_3 | 0 |
| S_4 | 0 |

The corrected word matches the codeword from section 7.2 exactly.

## 27.8 The decoder's final verdict

The second syndrome pass in chapter 3.2 is the mathematical check that the corrected word has zero syndromes. It is the last line of defense. Some patterns with more than $t$ errors still produce a locator of the right degree with the right number of roots; the degree and root checks pass, but the corrected word is not a codeword. The recomputed syndromes catch those miscorrections.

A block is uncorrectable when any of the following holds:

- $deg(\Lambda) > t$ or $deg(\Lambda) = 0$ after the solver. A locator of degree zero means no errors were found despite nonzero syndromes, which is impossible. A locator of degree greater than $t$ means the error pattern needs more parity than the code has.
- the number of roots found by Chien search differs from $deg(\Lambda)$. The fundamental theorem of algebra for this finite field says a degree-$\nu$ polynomial has exactly $\nu$ roots counting multiplicity; if fewer distinct roots are found, the error locators are not distinct, which is not a correctable error pattern.
- $\Lambda'(X^{-1}) = 0$ at a located root while $\Omega(X^{-1}) \neq 0$. The Forney formula divides by the derivative, so a zero derivative with nonzero evaluator is a structural failure.
- the corrected word's syndromes are not all zero. This catches the otherwise-undetectable miscorrections where the locator happens to have the right degree and root count but describes the wrong error pattern.

## 27.9 From math to hardware

*Table 7.15: Mathematical operations mapped to decoder blocks*

| Math step | Hardware block | Cross-reference |
|---|---|---|
| syndrome evaluation | syndrome unit | chapter 3.2 |
| key-equation solver | riBM (default) or Euclid/ME | chapter 3.3 |
| root finding | Chien search | chapter 3.2 |
| error magnitude | Forney stage | chapter 3.2 |
| correction + replay | block buffer / corrector | chapter 3.2 |
| final zero-syndrome check | second syndrome unit + status | chapter 3.2 |

This is the loop that chapter 3.2 describes as data flow: syndromes in, locator out, Chien and Forney walk the block, the corrector replays it, and the second syndrome unit signs the verdict.

# 28 The Parameters n, k, t, m

A Reed-Solomon code boils down to four integers. Change any of them and you change the field table, the generator polynomial, the block length, or the parity count. This section names the four, gives the exact relations between them, and points at the RTL parameter for each.

## 28.1 m: the symbol width

$m$ is the bit width of one symbol. It's also the exponent in the field size:

$$\left|GF\left(2^m\right)\right| = 2^m.$$

The nonzero elements are $2^m - 1$. In the RTL this is `SYMBOL_WIDTH`, default 8, so one symbol is one byte (`rtl/macro/rs_encoder_core.sv:66`). Picking $m$ also caps the block length: $n \leq 2^m - 1$.

## 28.2 n: the block length

$n$ is the number of symbols in one codeword. A full-length Reed-Solomon code has

$$n = 2^m - 1.$$

The RTL parameter is `N_SYMBOLS`, default $255$ when $m = 8$ (`rtl/macro/rs_encoder_core.sv:69`). Anything smaller is a shortened code. You get it by taking the full RS$\left(2^m - 1, 2^m - 1 - 2t\right)$ code and assuming the first $\left(2^m - 1 - n\right)$ data symbols are zero. There's no zero-fill hardware — the encoder and decoder just change `N_SYMBOLS` (`rtl/macro/rs_encoder_core.sv:55-57,69`). The Chien search absorbs the shortening offset in its starting value (`rtl/fub/chien_search.sv:30-31`).

## 28.3 k: the user data length

$k$ is the number of user data symbols per block. It's derived from $n$ and $t$:

$$k = n - 2t.$$

In the RTL `K_SYMBOLS` is computed as `N_SYMBOLS - 2*T_SYMBOLS` (`rtl/macro/rs_encoder_core.sv:74`). So the parity count is exactly $n - k = 2t$. That's a defining difference from binary BCH, where $n - k \leq mt$. BCH can use fewer parity symbols for the same $t$ because it only corrects single bits; RS has to correct an entire $m$-bit symbol. The BCH math-to-pseudocode page uses the same riBM module and is at `../../../../bch/docs/bch_has/ch07_understanding_the_math/05_every_stage_math_to_pseudocode.md`.

## 28.4 t: the error-correction capability

$t$ is how many symbol errors the decoder is guaranteed to fix. It costs $2t$ parity symbols, so $n-k=2t$, and the designed minimum distance is

$$d=2t+1.$$

The generator polynomial has $2t$ consecutive roots:

$$g(x)=\prod_{i=0}^{2t-1}\left(x-\alpha^{b+i}\right).$$

`T_SYMBOLS` is the RTL name, default 8 (`rtl/macro/rs_encoder_core.sv:68`), and the LFSR taps enforce those $2t$ roots (`rtl/fub/gf/gf_lfsr_encoder.sv:107-108`).

## 28.5 b: the first-root offset

$b$ shifts the generator's root set from $\alpha^0, \alpha^1, \ldots$ to $\alpha^b, \alpha^{b+1}, \ldots$. It doesn't change $n$, $k$, $t$, or the correction power — it only changes the parity-check matrix. The RTL default is `FIRST_ROOT = 0` (`rtl/macro/rs_encoder_core.sv:70`). Section 7.3's worked example uses narrow-sense $b = 1$, so its roots are $\alpha^1, \alpha^2, \alpha^3, \alpha^4$.

## 28.6 S: symbols per beat

$S$ is how many symbols travel in one clock cycle. It's set by the bus width:

$$S = \frac{DATA_W IDTH}{SYMBOL_W IDTH}.$$

The AXI-Stream wrapper defaults to one symbol per beat (`rtl/top/rs_encoder_axis4.sv:49,56`). The AXI4 job wrapper defaults to a 32-bit bus, so with $m = 8$ you get $S = 4$ (`rtl/top/rs_encoder_axi4.sv:52,59`). The encoder and decoder unroll their per-symbol steps across those $S$ lanes and use `i_count` to select the partial-beat result (`rtl/fub/gf/gf_lfsr_encoder.sv:158-169`). The number of beats per block is $\lceil k/S \rceil$ and $\lceil n/S \rceil$ (`rtl/top/rs_encoder_axi4.sv:125-131`).

## 28.7 Parameter summary

Table 7.16 collects the six names and their RTL defaults.

*Table 7.16: Reed-Solomon parameters and their RTL names*

| Symbol | RTL parameter | Meaning | Default (m=8) |
|---|---|---|---|
| $m$ | `SYMBOL_WIDTH` | bits per symbol | 8 |
| $n$ | `N_SYMBOLS` | symbols per codeword | 255 (or shortened) |
| $k$ | `K_SYMBOLS` | user data symbols | $n - 2t$ |
| $t$ | `T_SYMBOLS` | correctable symbol errors | 8 |
| $b$ | `FIRST_ROOT` | exponent of first generator root | 0 |
| $S$ | `SYMBOLS_PER_BEAT` | symbols per clock beat | bus dependent |

## 28.8 Built profiles

Table 7.17 shows the two shipped parameter sets: the unconstrained library default and the Genesys 2 board harness.

*Table 7.17: Two shipped Reed-Solomon profiles*

| Profile | $(n, k)$ | $t$ | $m$ | $b$ | $S$ | Notes |
|---|---|---|---|---|---|---|
| Library default | $(255, 239)$ | 8 | 8 | 0 | 1 | Full-length code (`rtl/macro/rs_encoder_core.sv:66-70`) |
| Genesys 2 board | $(252, 236)$ | 8 | 8 | 0 | 4 | RS $(255, 239)$ shortened by 3 symbols so $n$ and $k$ are multiples of the 32-bit beat (`projects/fpga-systems/Genesys2/ecc-ip/reed-solomon/build-loop/rtl/rs_loop_cfg_pkg.sv:10-13,26-37`) |

The board configuration is explicit about why it shortens: a 32-bit AXI-Stream pattern checker compares whole beats, so every beat must be full (`rs_loop_cfg_pkg.sv:10-13`). Setting `N_SYMBOLS = 252` achieves that with no extra hardware.

## 28.9 Exact relations

Table 7.18 lists the equations that tie the parameters together.

*Table 7.18: Parameter equations*

| Relation | Equation | Why it matters |
|---|---|---|
| Field order | $\left\|GF\left(2^m\right)\right\|=2^m$ | Every symbol fits in an $m$ -bit register. |
| Full length | $n \le 2^m - 1$ | Alpha has multiplicative order $2^m - 1$. |
| Parity count | $n - k = 2t$ | Exactly $2t$ parity symbols for RS; BCH can use fewer. |
| Designed distance | $d = 2t + 1$ | Guarantees correction of up to $t$ symbol errors. |
| Symbols per beat | $S = DATA_W IDTH / SYMB$ | Sets the datapath width and beat count. |

## 28.10 Choosing the parameters: a worked example

Suppose the channel corrupts at most one byte in every thirty, the datapath is 32 bits wide, and you want to correct 8 symbols. Then $m=8$ gives one symbol per byte and $S=32/8=4$. With $t=8$, the full code is RS$(255,239)$. Because the consumer needs whole 32-bit beats, shorten it by 3 symbols to RS$(252,236)$. The parity count is still $n-k=16=2t$, and the designed distance is still $d=17$. The only RTL changes are `N_SYMBOLS = 252` and the bus width; the LFSR, syndrome cells, solver, Chien search, and Forney evaluator stay exactly the same.

RTL Design Sherpa · Learning Hardware Design Through Practice  GitHub · Documentation Index · MIT License

---

# 29 Every Stage: Math to Pseudocode

Section 7.1 built the field, section 7.2 built the code, and section 7.3 walked a decode by hand. This page turns each stage into the pseudocode the RTL actually runs — equation first, then the registers, then the exact module and line numbers. The riBM key-equation solver is shared with the BCH decoder; its math-to-pseudocode page is at `../../../../bch/docs/bch_has/ch07_understanding_the_math/05_every_stage_math_to_pseudocode.md`.

*Table 7.19: From mathematics to hardware blocks*

| Math step | Hardware block | Cross-reference |
|---|---|---|
| parity generation | `gf_lfsr_encoder` / `rs_encoder_core` | section 7.5 below |
| syndrome evaluation | `syndrome_unit` | section 7.5 below |
| key-equation solve | `key_equation_solver_ribm` or `key_equation_solver_euclid` | section 7.3, section 7.5 below |
| root finding | `chien_search` | section 7.3, section 7.5 below |
| error magnitude | `forney_evaluator` | section 7.5 below |
| erasure combine | `rs_erasure_unit` | section 7.5 below |
| correction + recheck | `rs_decoder_core` | section 7.5 below |

## 29.1 Encoder (gf_lfsr_encoder, wrapped by rs_encoder_core)

The encoder divides the data polynomial by the generator $g(x) = \prod_{i=0}^{2t-1} (x - \alpha^{b+i})$ and appends the remainder as parity. That division is a $2t$-stage linear feedback shift register over $GF(2^m)$.

**The math.** For one input symbol $d$:

$$\begin{array}{ll} fb & d \oplus r[2t-1], \\ n[0] & fb \cdot g_0, \\ n[j] & r[j-1] \oplus fb \cdot g_j \quad (j = 1 \ldots 2t-1). \end{array}$$

The output codeword is the $k$ data symbols followed by the $2t$ remainder symbols.

**The pseudocode.**

```
g[0..2t-1] = generator coefficients, leading 1 implicit
r_reg[0..2t-1] = 0

function step_one(r, d):
    fb = d ^ r[2t-1]
    n[0] = fb * g[0]
    for j = 1 .. 2t-1:
        n[j] = r[j-1] ^ (fb * g[j])
    return n

# data phase: low lanes are the earlier symbols
for each input beat with i_count valid lanes:
    w_chain[0] = r_reg
    for u = 0 .. S-1:
        w_chain[u+1] = step_one(w_chain[u], lane[u])
    r_reg = w_chain[i_count]

# parity drain phase
for each drain beat:
    for u = 0 .. S-1:
        out_lane[u] = (u < 2t) ? r_reg[2t-1-u] : 0
    for j = 0 .. 2t-1:
        r_reg[j] = (j >= S) ? r_reg[j-S] : 0
```

**In the RTL.** `gf_lfsr_encoder.sv:142-156` implements `step_one`, `:158-169` unrolls $S$ steps and selects the partial-beat result, and `:171-175` drains the parity in transmission order. `rs_encoder_core.sv:147-187` sequences the data and drain phases around the skid buffer.

## 29.2 Syndrome unit (`syndrome_unit`, `gf_syndrome_cell`)

The syndrome unit evaluates the received word at each generator root. There are $2t$ roots, so there are $2t$ independent syndrome cells.

**The math.** For root $\alpha^{b+i}$:

$$S_i = R\left(\alpha^{b+i}\right) = \sum_j r_j \alpha^{(b+i)(n-1-j)}.$$

Each cell updates by Horner's rule as the symbols arrive:

$$s \leftarrow s \cdot \alpha^{b+i} \oplus r_j.$$

**The pseudocode.**

```
for i = 0 .. 2t-1:
    root_i = alpha^(b+i)
    r_s[i] = 0

for each received beat (first beat resets the accumulators):
    for i = 0 .. 2t-1:
        w_chain[i][0] = first_beat ? 0 : r_s[i]
        for u = 0 .. S-1:
            w_chain[i][u+1] = w_chain[i][u] * root_i ^ lane[u]
        r_s[i] = w_chain[i][i_count]

ow_synd[i]    = r_s[i]
ow_all_zero   = (all r_s == 0)
```

**In the RTL.** `syndrome_unit.sv:73-89` instantiates $2t$ `gf_syndrome_cell` instances, one per root. `gf_syndrome_cell.sv:76-88` is the $S$-step Horner chain, and `syndrome_unit.sv:91-92` forms the all-zero flag.

## 29.3 Key-equation solver riBM (`key_equation_solver_ribm`, `ribm_pe`)

The riBM solver inverts the key equation $\Lambda(x)S(x) \equiv \Omega(x) \left(mod\ x^{2t}\right)$ in exactly $2t$ cycles. It is the default solver selected by `KES_ALGO = "RIBM"` (`rtl/macro/rs_decoder_core.sv:123,335-357`). The same `key_equation_solver_ribm` module is instantiated by the BCH decoder.

**The math.** The algorithm keeps $3t+1$ processing elements. Each element holds a pair $(\Delta_i, \Theta_i)$. One iteration uses the discrepancy $\delta = \Delta_0$ and a swap flag:

$$\begin{aligned} \Delta_i^{\text{new}} &\quad \gamma \cdot \Delta_{i+1} \oplus \delta \cdot \Theta_i, \\ \Theta_i^{\text{new}} &\quad \text{swap}\ ?\ \Delta_{i+1} : \Theta_i. \end{aligned}$$

The control updates are:

$$\begin{aligned} \gamma &\quad \leftarrow \text{swap}\ ?\ \delta : \gamma, \\ k &\quad \leftarrow \text{swap}\ ?\ -k-1 : k+1. \end{aligned}$$

**The pseudocode.**

```
NPE = 3*t + 1
for i = 0 .. 2t-1:
    Delta[i] = Theta[i] = S[i]
for i = 2t .. 3t-1:
    Delta[i] = Theta[i] = 0
Delta[3t] = Theta[3t] = 1
gamma = 1
k = 0

repeat 2t times:
    delta = Delta[0]
    swap  = (delta != 0) && (k >= 0)
    for i = 0 .. NPE-1:
        Delta_next[i] = gamma*Delta[i+1] ^ delta*Theta[i]
        Theta_next[i] = swap ? Delta[i+1] : Theta[i]
    Delta = Delta_next
    Theta = Theta_next
    if swap:
        gamma = delta
        k = -k - 1
    else:
        k = k + 1

o_lambda = Delta[t .. 3t]      # Lambda_0 .. Lambda_2t
o_omega  = Delta[0 .. t-1]     # high-half evaluator, Omega_0 ..
Omega_{t-1}
```

```
o_deg    = degree of Lambda among the 2t+1 coefficients
o_deg_err = any Lambda_{t+1..2t} is nonzero
```

**In the RTL.** `key_equation_solver_ribm.sv:95` sets `NPE = 3t+1`. The array is loaded at `:137-145`, the processing elements are at `:147-164`, the swap logic is at `:132-134`, the control update is at `:180-188`, and the outputs are selected at `:194-213`. Each `ribm_pe` (`rtl/fub/gf/ribm_pe.sv:43`) performs the two multiplies and two register updates (`ribm_pe.sv:24-32,79-82`).

## 29.4 Key-equation solver Euclid (key_equation_solver_euclid)

The alternative solver is an inversionless extended Euclidean algorithm selected by `KES_ALGO = "EUCLID"`. It divides $x^{2t}$ by the syndrome polynomial until the remainder degree drops below $t$.

**The math.** Registers $R$ and $Q$ hold the two remainders top-aligned (leading coefficient at index $2t$). Registers $\tilde{\lambda}$ and $\tilde{\mu}$ hold the corresponding multipliers, also kept top-aligned. One cycle does one of:

- normalize $R$: if $R[2t]=0$, shift $R$ and $\tilde{\lambda}$ up by $x$, `degR--`;
- normalize $Q$: if $Q[2t]=0$, shift $Q$ and $\tilde{\mu}$ up by $x$, `degQ--`;
- cross: $R_{\text{new}}=(Q[2t]\cdot R \oplus R[2t]\cdot Q)\cdot x$, $\tilde{\lambda}_{\text{new}}=(Q[2t]\cdot\tilde{\lambda} \oplus R[2t]\cdot\tilde{\mu})\cdot x$, then swap roles if `degR < degQ`.

The loop stops when $\deg R<t$.

**The pseudocode.**

```
R[0..2t] = 0; R[2t] = 1                 # x^2t
Q[0..2t] = 0; for i=0..2t-1: Q[i+1] = S[i]   # S(x) top-aligned
lam_tilde[0..LW-1] = 0
mu_tilde[0..LW-1] = 0; mu_tilde[1] = 1
degR = 2t; degQ = 2t-1

while degR >= t:
    if R[2t] == 0:
        R       = shift_up(R)       # index i <- old i-1
        lam_tilde = shift_up(lam_tilde)
        degR -= 1
    elif Q[2t] == 0:
        Q       = shift_up(Q)
        mu_tilde = shift_up(mu_tilde)
        degQ -= 1
    else:
        a = R[2t]; b = Q[2t]
        Rn = (b*R) ^ (a*Q)
        Ln = (b*lam_tilde) ^ (a*mu_tilde)
        if degR < degQ:
            Q, mu_tilde, degQ = R, lam_tilde, degR
            degR = old_degQ - 1
        else:
            degR -= 1
        R       = shift_up(Rn)
        lam_tilde = shift_up(Ln)

sh = 2t - degR
o_lambda = lam_tilde[sh .. sh+2t]      # Lambda_0 .. Lambda_2t
```

```
o_omega  = R[sh .. sh+t-1]             # textbook evaluator
o_deg    = degree of Lambda
o_deg_err = degR out of range or high coefficients nonzero
```

**In the RTL.** `key_equation_solver_euclid.sv:118-126` declares the four register arrays and the two signed degrees. The initial values are loaded at `:212-225`, the normalize/cross/swap step is at `:150-199`, and the output un-shift is at `:270-299`.

## 29.5 Chien search (`chien_search`)

The Chien search evaluates the locator polynomial at every symbol position to find its roots. A root at position $j$ means that symbol is in error.

**The math.** Symbol $j$ (the $j$-th symbol transmitted) has location

$$X_j = \alpha^{n-1-j}.$$

It is in error when

$$\Lambda\left(X_j^{-1}\right) = 0.$$

The search keeps one cell per locator coefficient. Cell $i$ holds $\Lambda_i \cdot X_j^{-i}$ for the current beat's first position. It is initialized at position 0 with $X_0^{-i} = \alpha^{-i(n-1)}$ and stepped by $\alpha^{iS}$ per beat. The odd-index lane sum is

$$\text{odd_sum}[u] = \sum_{i\,\text{odd}} \Lambda_i \cdot \left(X_{j+u}\right)^{-i} = X_{j+u}^{-1} \cdot \Lambda'\left(X_{j+u}^{-1}\right),$$

which is the formal derivative handed to the Forney stage.

**The pseudocode.**

```
for i = 0 .. t:
    r_c[i] = Lambda_i * alpha^(-i*(n-1))

for base = 0, S, 2S, ... while base < n:
    for u = 0 .. S-1:
        sum = 0; odd = 0
        for i = 0 .. t:
            term = r_c[i] * alpha^(i*u)
            sum ^= term
            if i is odd: odd ^= term
        o_root[u]    = (sum == 0)
        o_odd_sum[u] = odd
    for i = 0 .. t:
        r_c[i] = r_c[i] * alpha^(i*S)
```

**In the RTL.** `chien_search.sv:87-109` initializes and steps the cells, and `:129-147` forms the per-lane sum and odd-sum outputs. The decoder core tracks the beat base position with `r_c_pos` (`rtl/macro/rs_decoder_core.sv:589-590,705-724`).

## 29.6 Forney evaluator (`forney_evaluator`)

The Forney stage turns each Chien root into the actual error magnitude.

**The math.** With the Chien search supplying $\text{odd_sum} = X^{-1}\Lambda'\left(X^{-1}\right)$, the error value is

$$e_j = \frac{X_j^{-(b+off)} \cdot \Omega\left(X_j^{-1}\right)}{\text{odd_sum}}.$$

Here's the part that trips people up: the offset is $off = 2t$ for the riBM solver and $off = 0$ for the Euclid solver and for erasure mode. riBM's `o_omega` is the *high half* of $S(x)\Lambda(x)$, coefficients $2t \ldots 3t-1$, not the textbook $\Omega = S\Lambda \bmod x^{2t}$. The extra $X^{-2t}$ factor in the numerator converts that high-half evaluator into the same ratio as the textbook form (`rtl/fub/forney_evaluator.sv:8-11,24-33`). The RTL doesn't use a $\Lambda_0$-normalization trick — it performs a real `gf_inv` per lane.

**The pseudocode.**

```
OFF = b + off        # off = 2t for riBM, 0 for Euclid / erasures
for i = 0 .. t-1:
    r_w[i] = Omega_i * alpha^(-(i+OFF)*(n-1))

for base = 0, S, 2S, ... while base < n:
    for u = 0 .. S-1:
        w_num[u] = 0
        for i = 0 .. t-1:
            w_num[u] ^= r_w[i] * alpha^((i+OFF)*u)
        w_den_inv[u] = inv(odd_sum[u])
        o_err_val[u] = w_num[u] * w_den_inv[u]
        o_den_zero[u] = (odd_sum[u] == 0)
    for i = 0 .. t-1:
        r_w[i] = r_w[i] * alpha^((i+OFF)*S)
```

**In the RTL.** `forney_evaluator.sv:84` computes `OFF = FIRST_ROOT + (OMEGA_HIGH_HALF ? 2t : 0)`, `:86-109` holds and steps the Omega cells, and `:130-152` evaluates each lane with one `gf_inv` and one `gf_mul`. `rs_decoder_core.sv:679-686` selects `OMEGA_HIGH_HALF` from the solver choice and erasure-support flag.

## 29.7 Erasure path (rs_erasure_unit)

When ERASURE_SUPPORT = 1, the decoder accepts symbols flagged as known bad. An erasure costs one parity symbol instead of two, so the correction bound is

$$2\mu + f \le 2t,$$

where $\mu$ is the number of unknown errors and $f$ is the number of erasures. The RTL enforces the equivalent budget $w_budget = (2t - f)/2$ (rtl/macro/rs_decoder_core.sv:497-504). Watch the boundary: $f = 2t$ is only valid when $\mu = 0$, because the budget drops to zero.

**The math.** The erasure locator is

$$\Gamma(x) = \prod_{p=1}^{f} (1 + X_p x),$$

where $X_p = \alpha^{n-1-p}$ is the location of the flagged symbol. The Forney syndromes are

$$GS = \Gamma \cdot S \bmod x^{2t}.$$

The solver window is stripped differently for the two algorithms: riBM receives $GS \gg f$ (the low $f$ coefficients dropped), while Euclid receives $x^f \cdot T$ (the same values shifted up). After solving, the combined polynomials are

$$\Lambda_c = \Gamma \cdot \Lambda_e, \Omega_c = \Lambda_e \cdot GS \bmod x^{2t}.$$

If $GS$'s high cells are all zero, the erasures alone explain every syndrome and the solver is bypassed with $\Lambda_e = 1$, so $\Lambda_c = \Gamma$ and $\Omega_c = GS$.

**The pseudocode.**

```
# Stage A: record flagged locations
r_xpow = alpha^(n-1)
r_xfile[0..2t] = 0
f = 0
for each received beat:
    for u = 0 .. S-1:
        if erasure[u] and u < count:
            r_xfile[f] = r_xpow * alpha^(-u)
            f += 1
    r_xpow = r_xpow * alpha^(-count)

# Stage B: TRANS
r_gam[0] = 1; r_gam[1..2t] = 0
r_gs[0..2t-1] = S[0..2t-1]
for cycle = 0 .. f-1:
    x = r_xfile[cycle]
    for i = 2t .. 1:
```

```
        r_gam[i] ^= x * r_gam[i-1]
    for j = 2t-1 .. 1:
        r_gs[j] ^= x * r_gs[j-1]
o_t_zero = (r_gs[f..2t-1] are all zero)

# solver input window
if KES_ALGO == "RIBM":
    kes_synd[i] = (i+f < 2t) ? r_gs[i+f] : 0
else:
    kes_synd[i] = (i >= f) ? r_gs[i] : 0

# Stage B: COMB (after solver returns Lambda_e of degree deg_e)
acc_lam[0..2t] = 0
acc_om[0..2t-1] = 0
for ci = deg_e .. 0:
    lc = Lambda_e[ci]
    for i = 2t .. 0:
        acc_lam[i] = (i>0 ? acc_lam[i-1] : 0) ^ lc * r_gam[i]
    for j = 2t-1 .. 0:
        acc_om[j] = (j>0 ? acc_om[j-1] : 0) ^ lc * r_gs[j]

o_lambda_c = o_t_zero ? r_gam : acc_lam
o_omega_c  = o_t_zero ? r_gs  : acc_om
o_deg_c    = min(deg_e + f, 2t+1)
```

**In the RTL.** Stage A records locations in `r_xfile` (`rs_erasure_unit.sv:141-205`). TRANS builds `r_gam` and `r_gs` (`rs_erasure_unit.sv:309-324`). The solver window is selected at `:326-344`. COMB multiplies through `Lambda_e` at `:347-412`, and the `o_t_zero` bypass is handled at `:52-54,285-288,398-412`.

## 29.8 Correction and verdict (`rs_decoder_core`)

The final stage replays the buffered received word, XORs each located error magnitude into the corresponding symbol, and asks a second syndrome unit whether the result is a valid codeword.

**The math.** Correction is addition in characteristic 2:

$$c_j = r_j \oplus e_j \text{ for each found root } j.$$

The block is uncorrectable when any of the following is true:

- the solver reported a degree error, or $deg(\Lambda) > t$ (or the erasure budget was exceeded);
- the number of Chien roots does not equal $deg(\Lambda)$;
- the Forney denominator is zero at a located root;
- the corrected word's syndromes are not all zero.

The final verdict expression is

$$w_uncorrectable_final = r_sv2_correct \wedge \left(r_sv2_bad|\right)\left(r_sv2_roots \neq r_sv2_deg|\right)r_sv2_den_zero|\right)!w_rechk_zer$$

**The pseudocode.**

```
# C1: walk one beat
for base = 0, S, 2S, ... while base < n:
    for u = 0 .. S-1:
        pos = base + u
        hit[u] = correct_en && valid[pos] && chien_root[u]
        corr[u] = hit[u] ? forney_val[u] : 0

# C2: replay and correct
for u = 0 .. S-1:
    w_c2_sym[u] = rx[u] ^ (hit[u] ? corr[u] : 0)

# second syndrome pass over the corrected stream
rechk_zero = syndromes_all_zero(w_c2_sym)

# root count, saturated
roots_final = r_c_roots + count(hit[0..S-1])

# verdict, one cycle after the last beat leaves C2
uncorrectable = r_sv2_correct &&
                (r_sv2_bad ||
                 (r_sv2_roots != r_sv2_deg) ||
                 r_sv2_den_zero ||
                 !rechk_zero)
```

**In the RTL.** `rs_decoder_core.sv:705-724` maps lanes to symbol positions and computes hits, `:756-759` XORs corrections, `:762-778` instantiates the second syndrome unit, `:781-784` counts roots, and `:790-794` forms the final uncorrectable verdict. The status ports `out_status_ok/uncorrectable/frame_err/corrected` are driven at `:836-847`.

## 29.9 Trace: RS(15,11) revisited

The script `ch07_math_trace.py` (in this directory) reproduces the section 7.3 example with the same field and the RTL-shaped algorithms above. It reads Table 7.9 as polynomial coefficient order — position $j$ is the coefficient of $x^j$ — and feeds the RTL routines in the reversed transmission order; see the convention note at the start of section 7.3's worked example. Its output is:

```
GF(2^4) primitive polynomial 0x13 = x^4 + x + 1
alpha order = 15; doc element vectors match

RS(15,11) t=2 b=1 received word (doc positions 0..14):
  0:alpha^0 1:alpha^6 2:alpha^1 3:alpha^12 4:0 5:alpha^0 6:alpha^1
7:alpha^4 8:alpha^2 9:alpha^8 10:alpha^5 11:alpha^13 12:alpha^3
13:alpha^14 14:alpha^9

syndromes S1..S4: alpha^4 alpha^6 alpha^5 alpha^0
riBM Lambda (degree 2): alpha^1 + alpha^6 x + alpha^0 x^2
riBM Omega: alpha^9 + alpha^0 x
Chien roots (doc positions): 3, 11
Forney magnitudes:
  j=3: alpha^5
  j=11: alpha^9
corrected word == original codeword: yes
post-correction syndromes: all zero

profile checks: RS(255,239) and RS(252,236) both have n-k=16, 2t=16,
d=17
ALL CHECKS PASSED
```

The riBM locator is `alpha^1 + alpha^6 x + alpha^0 x^2`. That's exactly `alpha^1` times the Peterson locator `1 + alpha^5 x + alpha^14 x^2` from section 7.3. Because Chien search looks for zeros, the common scale factor doesn't move the roots: they're still at positions 3 and 11. The Forney evaluator carries the same scale, and the `off = 2t` exponent converts the high-half riBM Omega into the same ratio as the textbook evaluator, so the magnitudes come out exactly `alpha^5` at `j = 3` and `alpha^9` at `j = 11`. The corrected word matches the original codeword, and the post-correction syndromes are all zero.